Chapter 1: Introduction to Reado.ai

1.1 Background and Context

The rapid proliferation of digital communication networks, electronic publishing platforms, and automated journalistic workflows has resulted in an exponential accumulation of unstructured textual data across the global information ecosystem. Readers, domain analysts, legal practitioners, and scientific researchers are routinely required to navigate voluminous corpuses of literature, including peer-reviewed manuscripts, regulatory filings, corporate white papers, and syndicated news streams. While the accessibility and democratization of digital documents have expanded significantly, human cognitive bandwidth, attention spans, and reading speeds have remained fundamentally bounded. This systemic divergence between information volume and human processing capacity leads directly to severe cognitive fatigue, superficial skim-reading practices, and an elevated probability of overlooking mission-critical nuances, technical qualifications, and evidentiary assertions within lengthy source texts.

Natural Language Processing (NLP) and Computational Linguistics have long treated automated document summarization as a foundational mechanism to alleviate this cognitive bottleneck. Historically, the summarization literature has bifurcated computational approaches into two primary operational paradigms: extractive summarization and abstractive summarization. Extractive frameworks approach the task as a combinatorial sequence-selection problem. Statistical and graph-theoretic algorithms—such as Term Frequency-Inverse Document Frequency (TF-IDF), TextRank, LexRank, and maximal marginal relevance scoring—assign numerical saliency weights to discrete sentence units extracted directly from the source material. Sentences exceeding a threshold score are assembled verbatim into an output digest. Although extractive pipelines inherently preserve grammatical correctness and prevent semantic drift, they consistently suffer from significant structural limitations. Because sentences are severed from their original discourse context, extracted outputs frequently exhibit severe discourse incoherence, dangling referents, ambiguous pronominal attachments, and unnecessary stylistic filler that degrades overall communicative efficiency.

In contrast, abstractive summarization mirrors human intellectual synthesis. Rather than performing simple sentence copying, an abstractive system reads the entire source context into continuous latent semantic vector representations, identifies latent conceptual hierarchies, and synthesizes entirely new sentences token-by-token. This approach enables dynamic paraphrasing, multi-sentence fusion, syntactic restructuring, and the systematic elimination of peripheral anecdotes. Driven by the advent of deep sequence-to-sequence neural architectures, recurrent encoder-decoder topologies, and attention-based Transformer models, abstractive text generation has emerged as the definitive standard for enterprise and academic document compression.

Despite these dramatic architectural breakthroughs, contemporary abstractive summarization deployments remain predominantly static and monolithic. Existing industrial application programming interfaces (APIs)

and pre-trained checkpoint baselines generate a single, unyielding summary of static length and uniform linguistic complexity for any given source document. Such static generation pipelines completely disregard user-specific cognitive constraints, variable operational environments, and targeted reading proficiencies. A postgraduate domain specialist investigating technical mechanics requires an analytical, structured synthesis that preserves technical nomenclature, quantitative figures, and causal linkages. Conversely, an executive or casual reader operating under extreme temporal limitations requires an immediate, plain-language digest stripped of recursive clauses and parenthetical discourse. A rigid, one-size-fits-all model inherently compromises across these divergent use cases, generating outputs that are either overly simplified for advanced research or unnecessarily dense for rapid decision-making. To overcome these limitations, modern natural language systems must transition toward controllable, adaptive text generation architectures.

1.2 Motivation: Digital Content Overload

Digital content overload is characterized by the structural imbalance between daily information acquisition demands and bounded biological reading thresholds. The modern digital workplace and academic landscape require continuous engagement with lengthy reports, policy documents, and dynamic media cycles. Consequently, knowledge workers spend a disproportionate share of productive hours filtering redundant descriptions rather than extracting actionable intelligence.

Standard digital interfaces provide passive reading environments. Readers are forced to manually alter their reading velocity or depend on fragmented scanning heuristics, which often leads to the omission of vital caveats, methodology conditions, and structural premises. Traditional computational summarization tools fail to resolve this condition due to three primary limitations:

1. *Agnosticism Toward Reader Context*: Commercial models process inputs uniformly, applying identical generation heuristics whether the intended recipient is a school student, a technical engineer, or an executive manager. The complete absence of parameter-level controllability leaves the consumer unable to modulate output density to match real-time reading goals.

2. *Fixed Compression Ratios*: Conventional generation pipelines apply static token budgets. They lack the architectural flexibility to transition dynamically from rapid, low-footprint distillations (under 40 words) to detailed analytical syntheses (100 to 200 words) without requiring the deployment and maintenance of distinct, disconnected neural pipelines.

3. *Monolithic Syntactic and Lexical Density*: Off-the-shelf generative checkpoints reflect the uniform lexical distributions of their underlying pre-training corpora (predominantly standard news wire copy). They are unable to synthesize plain, monosyllabic declarative statements on the one hand, or elevate stylistic vocabulary and complex multi-clause dependencies on the other.

These operational constraints motivate the design and implementation of Reado.ai—an intelligent, context-driven generative platform engineered to give readers explicit, interactive control over the verbosity, syntactic depth, and linguistic complexity of generated text summaries.

1.3 Problem Statement

The central engineering and algorithmic challenge addressed in this research can be formally defined as follows:

"How can an end-to-end natural language processing system be designed, calibrated, and deployed to dynamically generate contextually faithful, abstractive summaries conditioned across discrete complexity and verbosity tiers, while systematically mitigating generative hallucinations, circular degenerative repetitions, and operational inference latency within a responsive web architecture?"

This overarching research question decomposes into several core technical sub-challenges:

- *Controllable Discrete Generation*: Steering deep autoregressive sequence-to-sequence language models to strictly respect discrete token budgets and complexity constraints without collapsing syntactic fluency or corrupting grammatical dependencies.
- *Linguistic Tier Calibration*: Formulating reproducible decoding policies and adversarial loss constraints that govern vocabulary selection, clausal nesting, and information density across three distinct reader tiers (Easy, Hard, and Editorial).
- *Degenerative Mode Suppression*: Suppressing common generative artifacts inherent to autoregressive beam search decoding—specifically hallucinations, semantic drift, and circular n-gram repetition loops—during aggressive compression phases.
- *Full-Stack Systems Engineering*: Bridging the divide between high-parameter deep learning inference backends and client-side web interfaces to deliver low-latency, controllable text transformations within an intuitive user workflow**.**

1.4 Research Objective and Scopes

To address the defined problem statement, this dissertation pursues the following primary research and engineering objectives:

1. *Architect a Multi-Tier Controllable Text Generation Core*: Develop an abstractive summarization pipeline utilizing deep sequence-to-sequence Transformer architectures, introducing conditional decoding parameters and length-penalty regularizations to enforce explicit token constraints.
2. *Formulate and Calibrate the Tri-Level Complexity Architecture*: Establish three distinct, calibrated operational modes within a unified platform:

- Level 1 (Easy Mode - TL;DR): High-ratio semantic compression optimized for rapid cognitive assimilation, featuring short declarative syntax, reduced subordinate clause density, and everyday vocabulary.
- Level 2 (Hard Mode - Analytical): Intermediate compression calibrated for technical and academic consumption, retaining causal structures, quantitative data points, and domain dependencies within structured sentences.
- Level 3 (Editorial Mode - Publication-Grade): Moderate compression emphasizing stylistic depth, elevated lexicon, and comprehensive context retention suitable for professional publication standards.

3. *Eliminate Generative Pathologies*: Implement constrained decoding strategies—including beam search optimization, n-gram blocking mechanisms, and length penalties—to eliminate degenerate circular looping and minimize ungrounded factual hallucinations.
4. *Empirical Benchmarking and Metric Validation*: Quantify system performance using benchmark datasets through standard NLP evaluation suites, measuring lexical overlap (ROUGE-1, ROUGE-2, ROUGE-L), semantic fidelity, and readability metrics (Flesch Reading Ease, Flesch-Kincaid Grade Level) across all operational tiers.
5. *Develop and Integrate the Reado.ai Web Platform*: Engineer a production-ready, full-stack web application leveraging Next.js 15, Drizzle ORM, NeonDB serverless PostgreSQL, and an asynchronous GPU inference backend to provide users with real-time text input, dynamic tier switching, and persistent generation history.

Research Scope and Boundaries: The experimental boundaries of this project focus on English-language expository, academic, and journalistic prose. Input context lengths are bounded to 1,024 subword tokens, mapping directly to high-throughput neural inference environments optimized for consumer-grade GPU hardware.

1.5 Organization of the Report

The remainder of this dissertation is systematically organized into four major thematic parts across 17 chapters:

- *Part I: Foundations and Literature Review (Chapters 1–5)*: Introduces the core concepts of the Reado.ai platform, details the evolution from extractive heuristics to neural abstractive summarization, traces deep learning foundations from Recurrent Neural Networks to the Attention mechanism, analyzes Transformer model architectures, and reviews modern pre-trained sequence-to-sequence language models such as BART and T5.
- *Part II: Adversarial Formulation and Controllable Generation (Chapters 6–8)*: Presents the theoretical formulation of Generative Adversarial Networks for discrete text, establishes the mathematical and

algorithmic architecture of the Reado.ai Tri-Level Contextual Engine, and examines decoding strategies designed to mitigate degenerative loops and factual hallucinations.

- *Part III: Data Pipelines and Training Methodology (Chapters 9–11)*: Examines benchmark training corpora (CNN/DailyMail, XSum), explores subword tokenization strategies (Byte-Pair Encoding, SentencePiece), and documents the hardware setup, mixed-precision environments, and optimization schedules.
- *Part IV: Quantitative Evaluation and System Deployment (Chapters 12–17)*: Formulates linguistic readability metrics, presents empirical benchmark results and comparative tier evaluations, examines qualitative edge cases, outlines the software architecture of the Next.js 15 full-stack platform, and concludes with key contributions, limitations, and future research directions.

Chapter 2: Text Summarization and Generation Paradigms

2.1 The Spectrum of Text Summarization

Automated text summarization stands as one of the definitive benchmarks within Natural Language Processing (NLP) and Computational Linguistics. The primary objective is to take a source document $\mathcal{D}$ composed of a sequence of tokens or sentences and generate a concise surrogate text $\mathcal{S}$ that accurately retains the core factual propositions, thematic elements, and communicative intent of the original material while satisfying a designated length constraint $|\mathcal{S}| \ll |\mathcal{D}|$.

The theoretical spectrum of text summarization is traditionally structured around three core operational dimensions:

1. *Output Mechanics (Extractive vs. Abstractive)*: Extractive paradigms select, rank, and concatenate exact surface-level units directly from the source text. Abstractive paradigms encode the underlying semantic context into continuous latent representations and generate novel lexical tokens autoregressively.
2. *Contextual Scope (Single-Document vs. Multi-Document)*: Single-document summarization targets isolated narrative streams, scientific articles, or news reports. Multi-document summarization requires an algorithm to aggregate information across heterogeneous sources, resolve conflicting facts, eliminate cross-document redundancies, and synthesize a cohesive global perspective.
3. *User Conditioning (Generic vs. Query-Focused)*: Generic summarization produces an objective, generalized distillation of the primary themes contained in a text. Query-focused summarization conditions the condensation process on external search queries, user intents, or domain-specific constraints.

Within this taxonomic spectrum, contemporary research has migrated decisively from surface-level extractive selection toward deep abstractive generation. While extractive approaches provide hard guarantees against factual hallucinations, their inability to condense complex cross-sentence arguments creates fundamental barriers to readability and information density.

2.2 Extractive Summarization Techniques

Extractive summarization approaches the condensation task as a combinatorial sequence-selection problem. Rather than generating new natural language sentences, extractive systems evaluate individual sentences or clauses $s_i \in \mathcal{D}$ using mathematical, statistical, or structural importance metrics, subsequently extracting the top-k scoring units to construct the summary:

$$\mathcal{S}_{\text{ext}} = \arg \max_{\mathcal{S} \subset \mathcal{D}, |\mathcal{S}| \leq k} \sum_{s_i \in \mathcal{S}} \text{Score}(s_i) - \lambda \cdot \text{Redundancy}(\mathcal{S})$$

Historically, extractive methods evolved through several computational phases:

- *Frequency-Based and Statistical Models*: Early foundational systems, pioneered by Luhn and Edmundson, relied on lexical frequency distributions, positional weighting, and cue-word dictionaries. The integration of Term Frequency-Inverse Document Frequency (TF-IDF) allowed algorithms to weigh sentences based on the statistical distinctiveness of their constituent terms relative to a background corpus:

$$\text{TF-IDF}(w, d, \mathcal{C}) = \text{TF}(w, d) \times \log\left(\frac{|\mathcal{C}|}{|\{d' \in \mathcal{C} : w \in d'\}|}\right)$$

While computationally efficient, pure frequency models are blind to synonymy, polysemy, and grammatical dependencies.

- *Graph-Theoretic Centrality Algorithms*: Algorithms such as TextRank and LexRank conceptualized documents as continuous semantic networks. Sentences represent vertices V, and edge weights E_{ij} correspond to lexical or semantic similarity scores (such as cosine similarity over bag-of-words or TF-IDF vectors). Applying random-walk algorithms analogous to PageRank, the stationary probability distribution yields a centrality score for each sentence:

$$S(V_i) = (1 - d) + d \sum_{V_j \in \text{Adj}(V_i)} \frac{w_{ji}}{\sum_{V_k \in \text{Adj}(V_j)} w_{jk}} S(V_j)$$

Vertices with the highest convergence scores represent the most central informational nodes of the discourse.

- *Machine Learning and Feature Extraction*: Supervised extractive approaches treat the problem as a sequence-labeling binary classification task, where each sentence s_i receives a label $y_i \in \{0,1\}$. Support Vector Machines (SVMs), Naive Bayes classifiers, and early Recurrent Neural Networks (RNNs) evaluated feature vectors containing sentence length, relative paragraph position, presence of named entities, and overlap with document titles.

- *Neural Latent Selectors (BERTSUM)*: Modern extractive architectures utilize bidirectional Transformer encoders (such as BERT) fine-tuned with inter-sentence classification layers. By inserting specialized tokens (such as [CLS] tokens at the start of every sentence), the Transformer outputs a contextual sentence representation vector h_i, which passes through a sigmoid classification layer to predict extraction probabilities.

Structural Pathologies of Extractive Frameworks

Despite preserving factual accuracy by copying human-authored text verbatim, extractive summarization exhibits fundamental linguistic failure modes:

1. *Discourse Incoherence*: Sentences are extracted out of their native context, severing rhetorical transitions and resulting in disjointed, jarring reading flows.

2. *Dangling Co-References*: Extracted sentences frequently contain anaphoric pronouns ("he," "they," "this finding") without including the antecedent noun introduced in preceding, unselected sentences.

3. *Stylistic Redundancy and Low Compression Density*: Human writers naturally weave peripheral details into primary clauses. Extractive systems must take entire sentences, incorporating background filler, parenthetical qualifiers, and conversational anecdotes, thereby wasting limited word budgets on low-value context.

2.3 Abstractive Summarization Techniques

Abstractive summarization circumvents the structural rigidities of extractive selection by reframing summarization as a conditional language modeling and sequence-to-sequence translation task. Formally, given a source sequence $\mathrm{x} = (x_1, x_2, \ldots, x_M)$, the abstractive model parameters θ are trained to maximize the conditional log-likelihood of generating a target sequence $\mathrm{y} = (y_1, y_2, \ldots, y_N)$ where $N \ll M$:

$$\mathcal{L}(\theta) = \sum_{t=1}^{N} \log P\left(y_t \mid y_{<t}, \mathrm{x}; \theta\right)$$

Abstractive synthesis operates across several interconnected linguistic operations:

- *Paraphrasing and Sentence Fusion*: Condensing the core propositions of two or three sequential sentences into a single, cohesive, compound sentence.
- *Lexical Substitution*: Replacing complex domain explanations with concise, high-impact terminology or simpler vocabulary depending on target requirements.
- *Information Elimination*: Selectively pruning subordinate clauses, parenthetical remarks, and non-restrictive appositives while preserving the primary semantic predicate.

Evolution of Neural Sequence-to-Sequence Modeling

1. *Recurrent Encoder-Decoder Frameworks*: Early abstractive neural architectures utilized Long Short-Term Memory (LSTM) networks or Gated Recurrent Units (GRUs) to compress the input document into a single static hidden vector h_M, which a decoder network iteratively unrolled into output tokens. These models struggled severely with long-term dependencies, frequently suffering from catastrophic forgetting when processing inputs exceeding 100 words.

2. *Attention-Augmented Recurrent Models*: Bahdanau and Luong introduced dynamic attention mechanisms that allowed the decoder to compute alignment weights α_{ti} over all encoder hidden states at each decoding step t. This innovation enabled the network to dynamically focus on relevant sections of the source document during text synthesis.

3. *The Pointer-Generator Architecture*: To resolve the "out-of-vocabulary" (OOV) problem and combat factual hallucinations, See et al. introduced the Pointer-Generator network. The architecture computes

a generation probability $p_{\text{gen}} \in [0,1]$ at each time step, allowing the model to smoothly interpolate between generating a token from the vocabulary distribution P_{vocab} and copying a token directly from the source text via pointing:

$$P(w) = p_{\text{gen}} P_{\text{vocab}}(w) + (1 - p_{\text{gen}}) \sum_{i:w_i=w} a_i^t$$

This hybrid approach served as a foundational bridge between extractive fidelity and abstractive fluency.

4. *Pre-trained Denoising Autoencoders*: Modern abstractive summarization relies on large-scale self-supervised sequence-to-sequence Transformers, such as BART (Bidirectional and Auto-Regressive Transformers) and T5 (Text-to-Text Transfer Transformer). Pre-trained on multi-gigabyte textual corpora using corruptive noise functions (token masking, sentence deletion, document rotation, and text infilling), these models possess rich linguistic representations, world knowledge, and fluent generative capabilities.

2.4 Limitations of Conventional Fixed-Length Summaries

Despite the high syntactic fluency achieved by contemporary pre-trained language models, production-grade summarization deployments remain predominantly static. Commercial APIs and baseline systems typically generate a single summary of arbitrary, fixed length for any given document.

This static paradigm imposes several operational limitations:

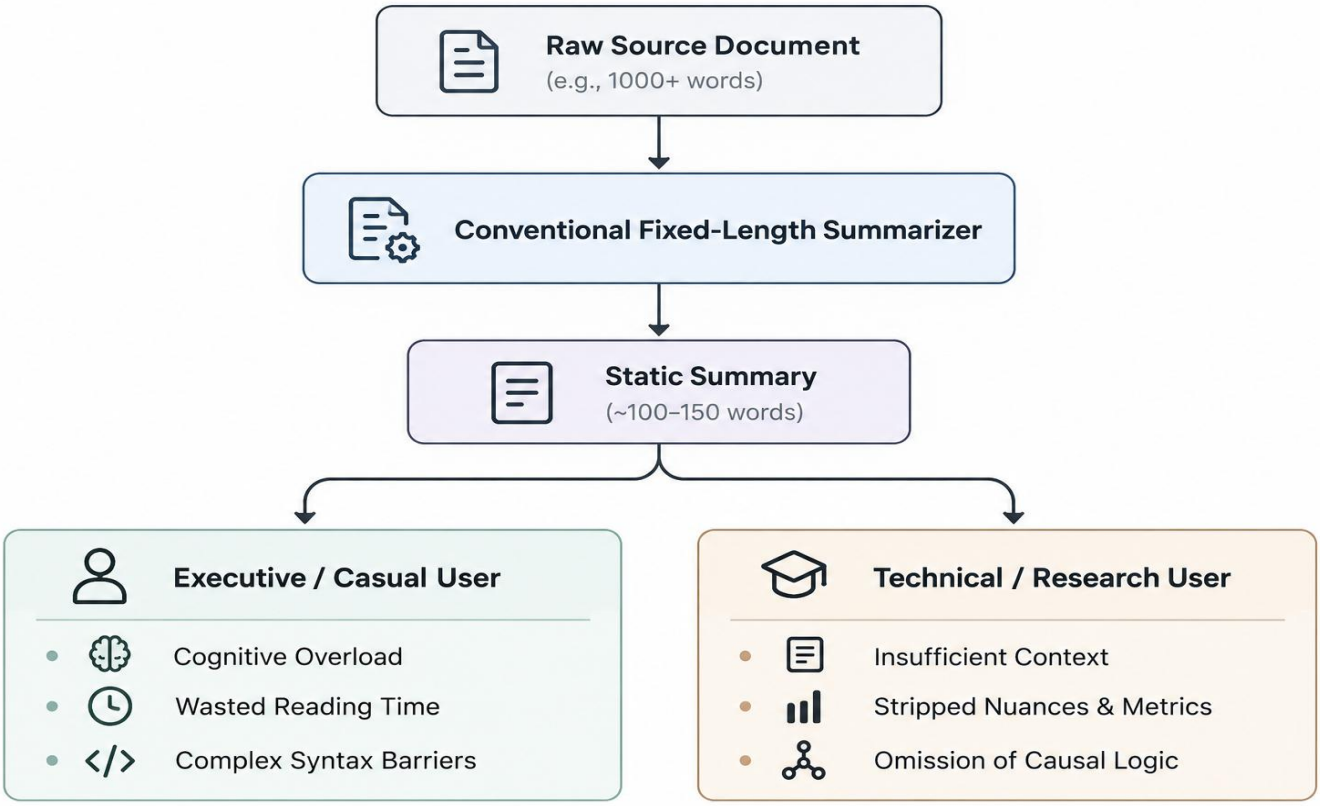


Figure 2.1: Limitations of Conventional Fixed-Length Summarization Pipelines

Agnosticism Toward Cognitive Bandwidth: Readers approach documents under fundamentally distinct constraints. A mobile user scanning news headlines requires an immediate, high-compression brief of under 30 words. An analyst or researcher digesting an experimental paper requires a 150-word synthesis that systematically outlines methodology, numerical metrics, and qualifications. A fixed-length output inevitably fails both users: it provides too much superficial detail for the former and discards essential evidentiary depth for the latter.

Exposure Bias and Degenerative Mode Collapse: Fixed-length generation models trained via standard maximum likelihood estimation (MLE) suffer from exposure bias. During autoregressive inference, small early decoding errors compound down the generation chain. If an unconditioned model is pushed to generate longer sequences than its standard checkpoint calibration, it regularly defaults to repetitive phrase looping, semantic drift, or hallucinations.

Monolithic Lexical Density: Standard summarization checkpoints reflect the uniform lexical and syntactic distributions of their primary training benchmarks (such as news wire articles). They lack the structural agility to shift between plain, low-Flesch-Kincaid monosyllabic constructions for non-specialists and sophisticated, multi-clause syntactic structures for academic or editorial dissemination.

2.5 The Need for Dynamic, Controllable Generation

To resolve the structural shortcomings of static summarization, modern NLP must advance toward **Controllable Text Generation (CTG)**. Controllable summarization decouples the synthesis process from static decoding policies, enabling users to steer the generation trajectory across specific dimensions: length, style, vocabulary complexity, and information density.

Dynamic controllability transforms the summarization system from a rigid black-box pipeline into an interactive cognitive tool:

- *Parameter-Driven Length and Verbosity Control*: Rather than accepting arbitrary stopping criteria dictated by the end-of-sequence (</s>) token, the decoding architecture allows users to modulate minimum and maximum length bounds alongside length-penalty parameters (α):

$$S(Y, X) = \frac{\log P\,(Y \mid X)}{(5 + |Y|)^{\alpha}/(5+1)^{\alpha}}$$

By adjusting α, the decoding beam can aggressively prioritize dense semantic compression or incentivize prolonged, comprehensive context retention.

- *Lexical and Syntactic Complexity Modulation*: Controllable generation empowers the underlying model to adapt its vocabulary and syntactic structures to match the target reading audience. Synthesizing summaries across discrete reading levels—ranging from elementary syntactical

structures to publication-grade editorial prose—demands systematic control over vocabulary distributions, subordinate clause depth, and grammatical complexity.

- *Operational Architecture of Reado.ai*: Reado.ai addresses this imperative through its tri-level complexity architecture, categorizing controllable generation into three calibrated operational modes:

  i. **Easy (TL;DR):** High-ratio semantic compression (>85%) optimized for rapid assimilation using simplified, direct declarative phrasing.

  ii. **Hard (Analytical):** Intermediate compression (70%–80%) designed for technical and academic consumption, retaining multi-clause dependencies, numerical metrics, and structural arguments.

  iii. **Editorial (Publication-Grade):** Moderate compression (50%–60%) tailored for professional dissemination, maintaining stylistic richness, narrative continuity, and sophisticated vocabulary.

Chapter 3: Neural Networks and Deep Learning in NLP

3.1 Recurrent Neural Networks (RNNs) and LSTMs

Prior to the advent of attention-centric neural modeling, automated natural language processing and sequence transduction tasks relied predominantly on Recurrent Neural Networks (RNNs). Unlike feed-forward networks that treat input features as statistically independent entities, an RNN is explicitly formulated to process sequential data $\mathbf{x} = (x_1, x_2, \ldots, x_T)$ by maintaining an internal continuous hidden state vector $\mathbf{h}_t \in \mathbb{R}^d$. At each discrete temporal step t, the recurrent transition function updates this internal state by combining the current input vector with the activation vector from the preceding time step:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$$

Where $\mathbf{W}_{hh} \in \mathbb{R}^{d \times d}$ denotes the hidden-to-hidden recurrent transition matrix, $\mathbf{W}_{xh} \in \mathbb{R}^{d \times d_{in}}$ represents the input-to-hidden weight projection matrix, and $\mathbf{b}_h$ is the corresponding bias term.

The Problem of Exploding and Vanishing Gradients

Training standard recurrent networks requires backpropagation through time (BPTT). Computing the gradient of the scalar loss $\mathcal{L}$ with respect to the recurrent parameter matrix $\mathbf{W}_{hh}$ necessitates computing the product of Jacobian matrices across temporal sequences:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{T} \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \left(\prod_{j=k+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} \right) \frac{\partial \mathbf{h}_k}{\partial \mathbf{W}_{hh}}$$

As the temporal distance $t - k$ increases, the continuous matrix product $\prod_{j=k+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}$ causes gradient values to scale exponentially according to the spectral radius of the transition matrix. If the largest eigenvalue of $\mathbf{W}_{hh}$ is strictly less than unity, gradients vanish exponentially toward zero, preventing the network from updating weights based on early contextual cues. Conversely, if the eigenvalue exceeds unity, gradients explode, destabilizing weight convergence and precipitating numerical overflow.

Long Short-Term Memory (LSTM) Networks

To counteract the vanishing gradient pathology, Hochreiter and Schmidhuber formulated the Long Short-Term Memory (LSTM) cell architecture. LSTMs introduce an additive internal memory conduit called the cell state ($\mathbf{c}_t$), regulated by three multiplicative gating structures parameterizing information flow:

- *Forget Gate:* Determines what proportion of the historical cell state should be discarded:

$$\mathbf{f}_t = \sigma\big(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f\big)$$

- *Input Gate and Candidate State:* Determines what novel information should be assimilated into the internal memory state:

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$

- *Cell State Update:* Executes additive linear state updating, preserving uninterrupted gradient backpropagation conduits across long sequence spans:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

- *Output Gate and Hidden Activation:* Emits the revised hidden representation passed to subsequent operational layers:

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

Where $\sigma(\cdot)$ denotes the element-wise logistic sigmoid activation, and $\odot$ represents the Hadamard element-wise vector product. While LSTMs extended memory horizons relative to standard vanilla RNNs, their sequential processing dynamics continued to present fundamental computational constraints during large-scale document modeling.

3.2 Sequence-to-Sequence (Seq2Seq) Frameworks

Sutskever et al. and Cho et al. formalized the Sequence-to-Sequence (Seq2Seq) neural architecture, enabling neural systems to map an input sequence of variable length $\mathbf{x} = (x_1, x_2, \ldots, x_M)$ to an output sequence of independent length $\mathbf{y} = (y_1, y_2, \ldots, y_N)$. This paradigm serves as the foundational architectural blueprint for neural machine translation, abstractive summarization, and conditional text generation.

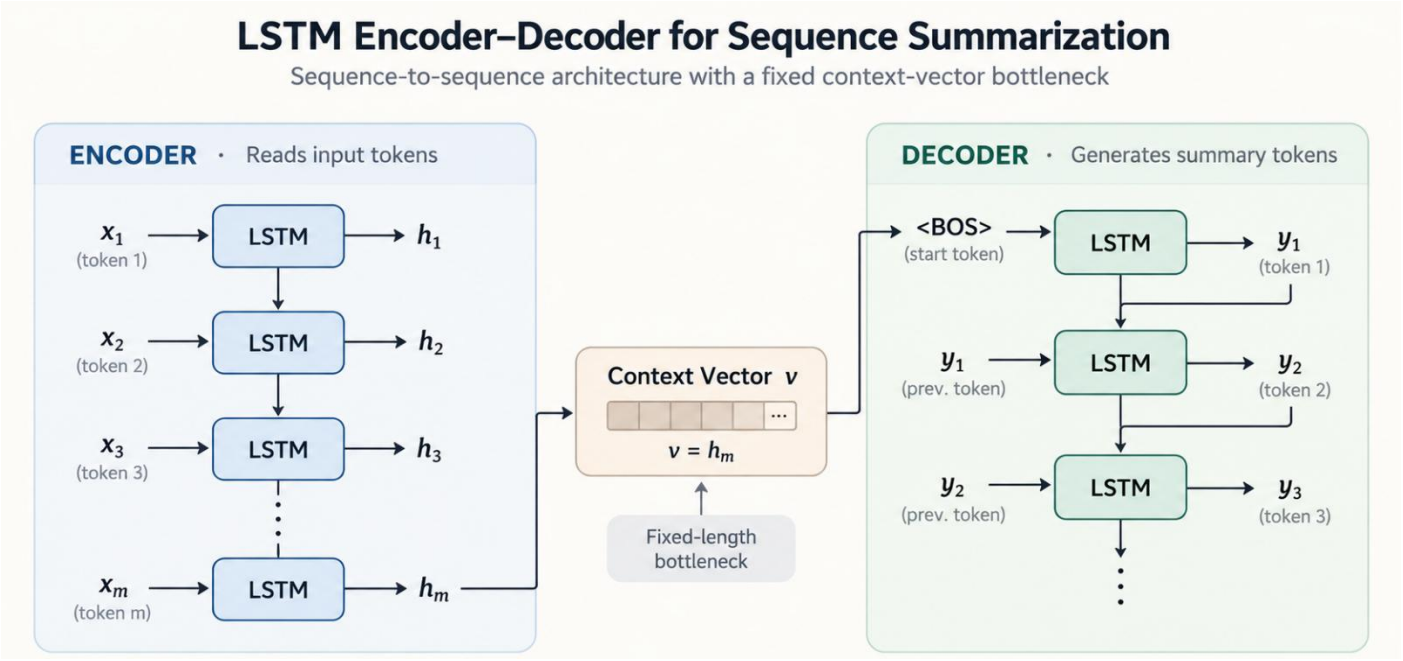


Fig 3.1: LSTM Encoder-Decoder for Seq2Seq Summarization

The canonical Seq2Seq framework comprises two asymmetric neural networks:

3.2.1 The Recurrent Encoder

The encoder sequentially consumes input tokens $x_m \in \mathbf{x}$. At each step, it updates its recurrent hidden state vector $\mathbf{h}_m$. Once the terminal token x_M is processed, the encoder emits its final hidden activation vector $\mathbf{h}_M$. This terminal vector is designated as the global context vector $\mathbf{v}$:

$$\mathbf{v} = \mathbf{h}_M = \text{EncoderRNN}(x_1, x_2, \dots, x_M)$$

The vector $\mathbf{v}$ is presumed to act as a complete, dense, fixed-dimensional semantic summary capturing the entirety of the input document's thematic context.

3.2.2 The Autoregressive Decoder

The decoder is a conditioned language generation model initialized using the context vector emitted by the encoder:

$$\mathbf{s}_0 = \mathbf{v}$$

At each output generation step t, the decoder maintains an internal hidden state $\mathbf{s}_t = \text{DecoderRNN}(\mathbf{s}_{t-1}, y_{t-1}, \mathbf{v})$. The probability distribution over the target vocabulary space $\mathcal{V}$ is derived by passing the hidden state through a linear projection layer parameterized by matrix $\mathbf{W}_s \in \mathbb{R}^{|\mathcal{V}| \times d}$ and normalizing via the Softmax function:

$$P(y_t \mid y_{<t}, \mathbf{x}) = \text{softmax}(\mathbf{W}_s \mathbf{s}_t + \mathbf{b}_s)$$

Training is conducted using teacher-forcing, where ground-truth tokens y^*_{t-1} are provided as inputs at each time step to maximize the cumulative sequence log-likelihood objective:

$$\mathcal{L}_{\text{Seq2Seq}}(\theta) = -\sum_{t=1}^{N} \log P\left(y_t^* \mid y_{<t}^*, \mathbf{x}; \theta\right)$$

3.3 The Information Bottleneck Problem

The primary theoretical and practical limitation of the classical Seq2Seq architecture is the Information Bottleneck Problem. In standard formulations, the model requires the encoder to compress an input document of arbitrary length, structural complexity, and conceptual density into a single, fixed-dimensional hidden vector $\mathbf{v} \in \mathbb{R}^d$.

This structural compression creates critical operational failures when applied to long-form document summarization:

1. *Capacity Saturation and Catastrophic Forgetting*: Vector spaces of fixed dimension d (typically $d \in [256,1024]$) possess finite informational entropy capacity. As source text length scales beyond 50 to

100 tokens, the continuous recurrent updates overwrite early network activations. The model experiences recency bias, preserving information situated near the end of the document while losing foundational premises introduced in early paragraphs.

2. *Loss of Fine-Grained Evidentiary Detail*: High-precision data points—such as numerical values, named entities, technical parameters, and conditional caveats—are lost during global vector compression. Consequently, the decoder is forced to hallucinate missing factual links or generate generic, non-committal phrasing during autoregressive decoding.

3. *Impeded Gradient Propagation*: During backpropagation through time, gradient signals emitted from the decoder must travel through the single context vector bridge $\mathbf{v} = \mathbf{h}_M$ before reaching early encoder states $\mathbf{h}_1, \ldots, \mathbf{h}_k$. This structural bottleneck compounds the gradient attenuation problem, hindering convergence across deep encoder networks.

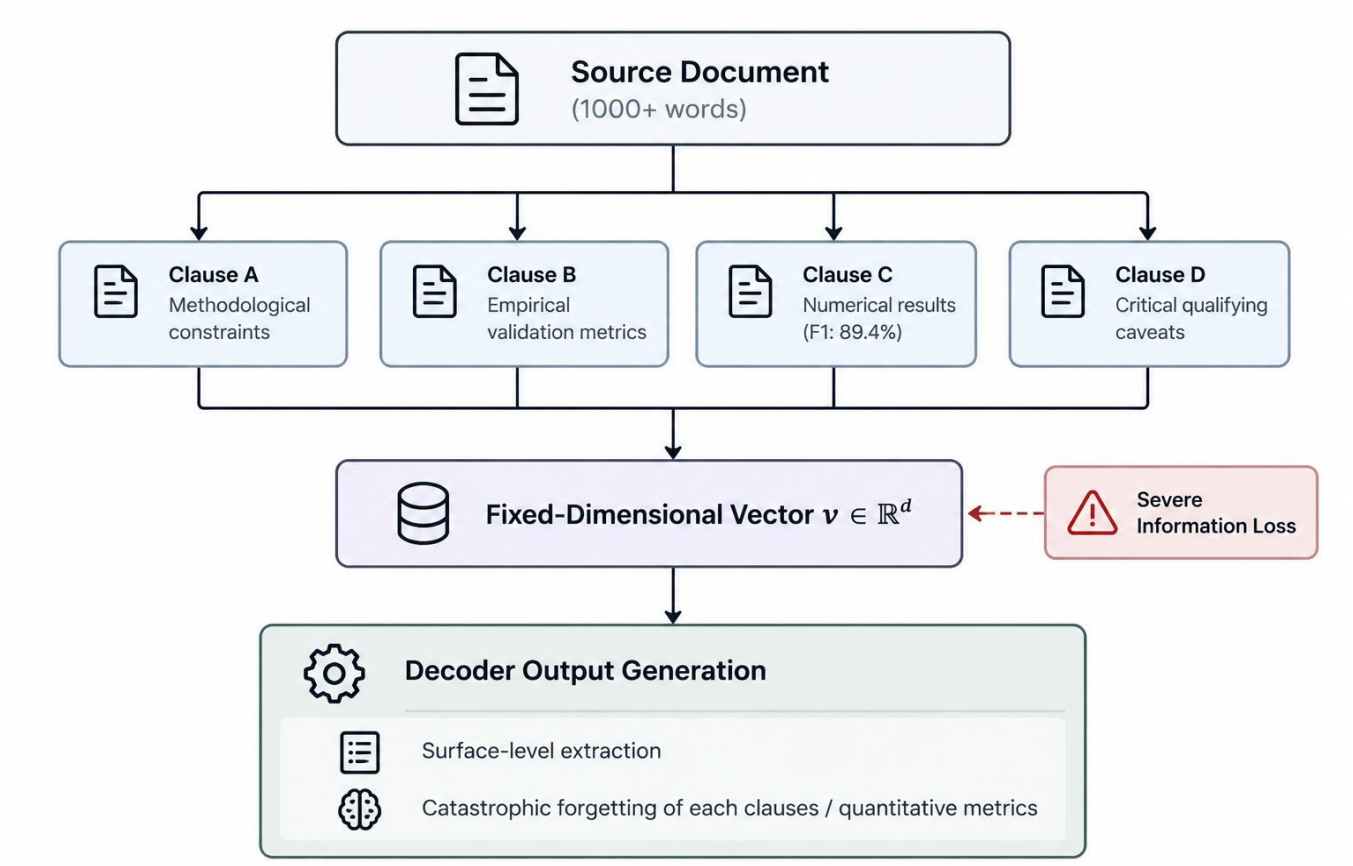


Fig 3.2: Information Loss in the Seq2Seq Encoder–Decoder Architecture

3.4 Additive Attention (Bahdanau Architecture)

Bahdanau et al. formulated additive attention by passing the current decoder hidden state $\mathbf{s}_{t-1}$ and each source encoder state $\mathbf{h}_i$ through a learned feed-forward neural layer with a hyperbolic tangent non-linearity:

$$e_{ti} = \mathbf{v}_a^\top \tanh(\mathbf{W}_a \mathbf{s}_{t-1} + \mathbf{U}_a \mathbf{h}_i)$$

Where $\mathbf{W}_a \in \mathbb{R}^{k \times d}$, $\mathbf{U}_a \in \mathbb{R}^{k \times d}$, and $\mathbf{v}_a \in \mathbb{R}^k$ are learned projection parameters. The scalar energy scores e_{ti} represent the raw semantic compatibility between the target decoding position and the source token at index i.

The alignment coefficients α_{ti} are obtained by normalizing the energy scores over the entire source document length using the Softmax function:

$$\alpha_{ti} = \frac{\exp(e_{ti})}{\sum_{k=1}^{M} \exp(e_{tk})}$$

The dynamic context vector $\mathbf{c}_t$ is computed as the probability-weighted expectation over all encoder hidden states:

$$\mathbf{c}_t = \sum_{i=1}^{M} \alpha_{ti} \, \mathbf{h}_i$$

The context vector $\mathbf{c}_t$ is then concatenated with the decoder hidden state $\mathbf{s}_t$ to derive the attentional hidden state $\tilde{\mathbf{s}}_t$, which directly informs the final output token distribution:

$$\tilde{\mathbf{s}}_t = \tanh(\mathbf{W}_c[\mathbf{c}_t; \mathbf{s}_t])$$

$$P(y_t \mid y_{<t}, \mathbf{x}) = \text{softmax}(\mathbf{W}_s \tilde{\mathbf{s}}_t)$$

3.4.2 Multiplicative Attention (Luong Architecture)

Luong et al. simplified the alignment scoring function by proposing multiplicative (dot-product) formulations, eliminating the intermediate feed-forward non-linearity in favor of direct matrix multiplication:

$$\text{Score}(\mathbf{s}_t, \mathbf{h}_i) = \begin{cases} \mathbf{s}_t^\top \mathbf{h}_i & \text{(Dot Product)} \\ \mathbf{s}_t^\top \mathbf{W}_a \mathbf{h}_i & \text{(General Bilinear Multiplicative)} \end{cases}$$

The multiplicative approach provides distinct computational advantages over additive scoring:

- **Matrix Multiplication Efficiency:** Multiplicative scoring can be formulated entirely as matrix multiplications ($\mathbf{S}\mathbf{W}_a\mathbf{H}^\top$), allowing implementations to exploit optimized Basic Linear Algebra Subprograms (BLAS) routines on GPU tensor cores.
- **Direct Gradient Paths:** Removing the hyperbolic tangent activation layer provides a direct linear gradient pathway from the output prediction loss back to intermediate encoder representations, accelerating model convergence.

The development of additive and multiplicative attention mechanisms fundamentally decoupled sequence generation from static recurrent memory limits. This architectural breakthrough laid the theoretical foundation for self-attention and the Transformer architectures analyzed in subsequent chapters.

Chapter 4: Transformer Architecture Overview

4.1 Transformer Model Architecture

The introduction of the Transformer model by Vaswani et al. replaced traditional recurrent neural networks with an architecture built entirely on self-attention mechanisms. By eliminating sequential recurrent processing, the model handles input sequences in parallel, dramatically improving training throughput on GPU accelerators and resolving the long-range dependency problem in NLP tasks.

The canonical Transformer relies on an asymmetric encoder-decoder topology. The encoder processes the source text bidirectionally to create contextualized vector representations, while the autoregressive decoder consumes these representations alongside previously generated tokens to predict the summary one token at a time.

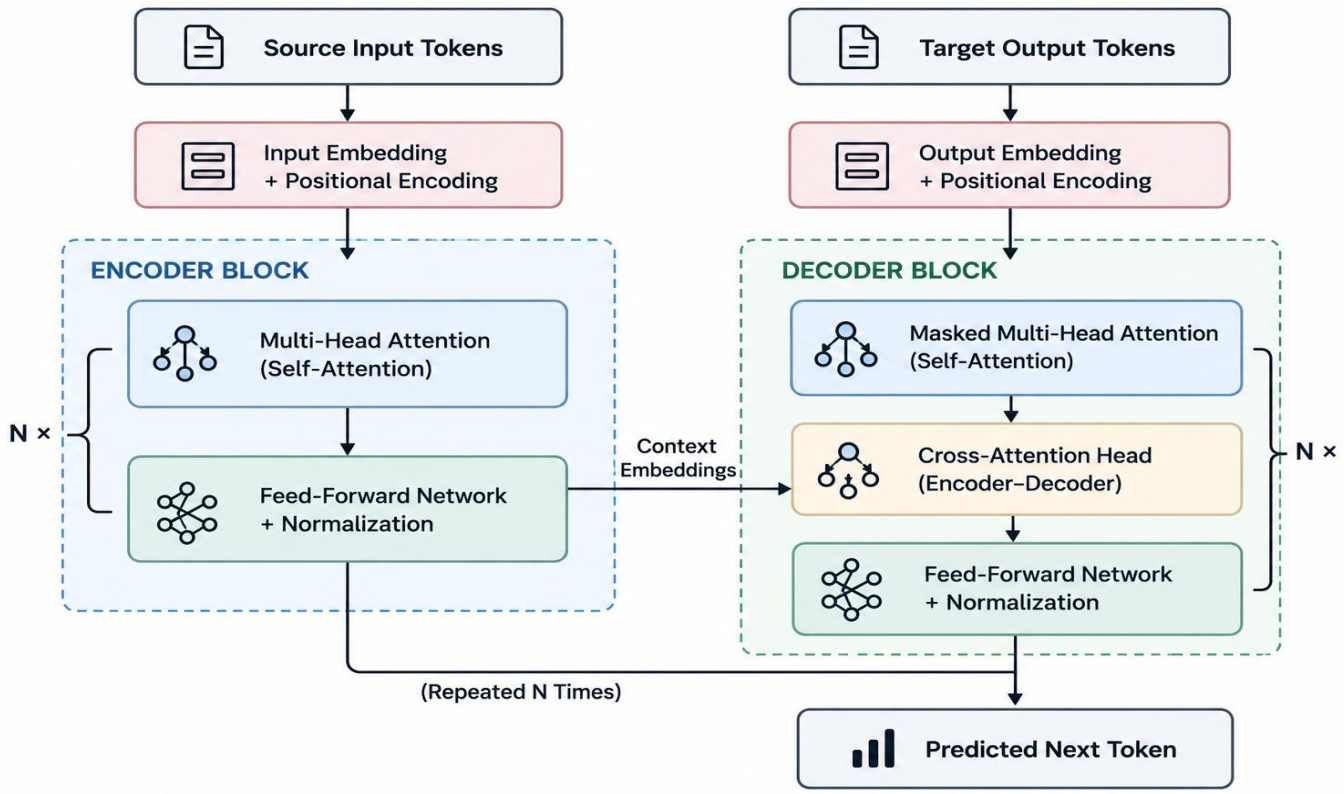


Fig 4.1: Transformer Encoder-Decoder Architecture

4.2 Scaled-Dot Product Attention

Scaled dot-product attention computes content-based alignment using three conceptual vectors: Queries (Q), Keys (K), and Values (V).

- Query (Q): The current token seeking context.
- Key (K): Candidate tokens offering contextual anchors.
- Value (V): The underlying semantic information stored across those tokens.

The dot product of Q and K measures semantic similarity. To prevent large vector dimensions from driving the softmax function into regions with near-zero gradients, the dot product is divided by a scaling factor proportional to the vector dimensionality ($\sqrt{d_k}$). The softmax distribution then weights the Value vectors to form the final contextual output.

Component	Operational Role	Vector Representation
Query (Q)	Probes the sequence to identify relevant semantic dependencies.	Current token embedding projection
Key (K)	Acts as an addressable index against which queries match.	Document token representation
Value (V)	The actual semantic payload retrieved during attention.	Unmasked information content
Scaling Factor ($\sqrt{d_k}$)	Stabilizes gradients by dampening extreme dot products.	Scalar dimensionality constant

Table 4.1: Attention Parameter in Scaled-Dot Product

4.3 Multi-Head Attention Mechanisms

A single attention operation risks averaging multiple distinct grammatical and semantic signals across the input. The Transformer addresses this by introducing Multi-Head Attention, which splits the hidden representations into parallel subspaces.

Each head projects Q, K, and V independently, allowing the network to capture distinct contextual patterns simultaneously:

- Syntactic Dependencies: Tracking local grammatical patterns, such as subject-verb agreement.
- Coreference Resolution: Matching pronouns to earlier nouns across long distances.
- Global Thematic Alignment: Evaluating high-level relationships across paragraph boundaries.

The outputs from all individual attention heads are concatenated and linearly projected back to the primary model dimension, producing a unified representation.

4.4 Positional Encodings & Feed-Forward Sublayers

Because self-attention is naturally order-agnostic, explicit positional information is required to preserve syntax, order, and reading flow.

4.4.1 Positional Encodings

The Transformer injects positional information by adding distinct frequency patterns to the initial token embeddings. Sine and cosine functions map coordinates across varying wavelengths, allowing the network to distinguish whether a word appears at the start, middle, or end of a sequence without using recurrence.

4.4.2 Feed-Forward Sublayers

Following the attention calculations, each Transformer layer includes a position-wise feed-forward network applied to each token position independently and identically. This module expands the representation space before projecting it back, deepening the network's capacity to synthesize abstract features. Each sublayer is surrounded by a residual connection followed by layer normalization, which stabilizes gradient propagation throughout deep network stacks.

4.5 Encoder v/s Decoder Representation

The Transformer partitions sequence transduction between two specialized structural components: the encoder stack and the decoder stack.

Structural Metric	**Encoder Block**	**Decoder Block**
Attention Scope	Full Bidirectional (Tokens see past and future)	Causal Unidirectional (Tokens only see the past)
Masking Policy	Padding mask only	Padding mask + Future lookahead mask
Sublayers per Block	2 (Self-Attention + Feed-Forward)	3 (Masked Self-Attn + Cross-Attn + Feed-Forward)
Functional Role	Encodes input document context	Generates summary tokens step-by-step
Cross-Attention Role	Generates Key (K) and Value (V) sources	Submits Query (Q) against encoder representations

Table 4.2: Difference between Representations of Encoder and Decoder

Chapter 5: Pre-Trainer Language Model for Text Generation

5.1 The Pre-training and Fine-tuning Paradigm

The pre-training and fine-tuning paradigm represents a fundamental shift in modern Natural Language Processing. Historically, neural architectures were trained from scratch on task-specific, human-annotated datasets. This classical supervised approach was severely constrained by the high cost and scarcity of labeled data, frequently causing deep networks to overfit when applied to complex sequence transduction tasks such as abstractive summarization.

The modern paradigm decouples representation learning into two distinct phases. During self-supervised pre-training, a deep neural model is exposed to vast, not curated textual corpora spanning hundreds of gigabytes, such as digitized books, news archives, and web crawls. Instead of relying on manual labels, the network learns the foundational mechanics of human language—including syntactic dependencies, semantic associations, world facts, and long-range contextual cues—by solving self-supervised surrogate tasks, such as predicting masked tokens or reconstructing corrupted text spans.

Once pre-training establishes a generalized language foundation, the model enters the fine-tuning phase. In this step, the network weights are initialized with the pre-trained parameters and adjusted using a smaller, task-specific dataset. For abstractive summarization, the model is supervised on document-summary pairs (such as CNN/DailyMail or XSum) using sequence cross-entropy loss. Fine-tuning aligns the model's rich latent linguistic representations with specific operational requirements, significantly reducing convergence time, improving factual fidelity, and yielding higher ROUGE scores compared to models trained from scratch.

5.2 Encoder-Decoder Models: T5 and BART

Sequence-to-sequence abstractive summarization requires an architecture capable of deeply understanding an arbitrary input passage while simultaneously generating a fluent, grammatically sound compression. Pre-trained encoder-decoder models are specifically engineered for this dual challenge, combining the contextual comprehension of bidirectional encoders with the generative fluency of autoregressive decoders.

The Text-to-Text Transfer Transformer (T5), developed by Raffel et al., frames every natural language processing task into a unified string-to-string format. Whether the task is classification, translation, question answering, or document summarization, T5 consumes a natural language task prefix (such as "summarize: ") alongside the source passage and generates the target text. T5 is pre-trained on the Colossal Clean Crawled Corpus (C4) using a span-corruption objective, where consecutive sequences of input tokens are masked with unique sentinel tokens and the decoder learns to reconstruct the missing spans. This text-to-text formulation provides architectural consistency and allows seamless transfer learning across disparate language tasks.

Bidirectional and Auto-Regressive Transformers (BART), formulated by Lewis et al., combines the bidirectional encoding capabilities of BERT with the autoregressive decoding structure of GPT. BART is

explicitly designed as a denoising autoencoder. During pre-training, input texts are corrupted using a diverse suite of noise transformations, including random token masking, token deletion, sentence permutation, document rotation, and text infilling via Poisson-distributed span lengths. The bidirectional encoder processes the corrupted document, while the causal autoregressive decoder learns to reconstruct the original, uncorrupted text. This denoising objective makes BART effective for abstractive summarization, as the model naturally learns to correct distorted syntactic flow, condense redundant clauses, and reorder information into coherent narrative summaries. Reado.ai leverages the "*facebook/bart-large-cnn"* checkpoint as its generative core due to its proven abstractive fidelity and balanced inference footprint.

Model	**Architectural Paradigm**	**Pre-training Objective**	**Primary Strength in Summarization**
BART	Sequence-to-Sequence (Bidirectional Encoder + Causal Decoder)	Denoising Autoencoding (Token masking, span infilling, sentence shuffling)	Exceptional structural paraphrasing and sentence reconstruction
T5	Sequence-to-Sequence (Unified Text-to-Text)	Span Corruption (Reconstructing missing multi-token spans marked by sentinels)	Multi-task versatility and cross-domain zero-shot adaptability
GPT-Style	Decoder-Only (Causal Autoregressive)	Next-Token Prediction (Unidirectional causal language modeling)	High conversational fluency and open-ended generative versatility

Table 5.1: Difference between T5 and BART Encoder-Decoder Models

5.3 Casual Language Models: GPT Architecture

Causal language models, epitomized by the Generative Pre-trained Transformer (GPT) series developed by Radford, Brown, et al., rely exclusively on a decoder-only architecture. Unlike encoder-decoder systems that dedicate separate parameters to document comprehension and sequence generation, GPT models process all text sequentially through stacked masked self-attention blocks. Every token can only attend to its historical left-hand context, completely preventing bidirectional cross-token interaction during initial document ingestion.Pre-training for causal language models is governed entirely by standard autoregressive next-token prediction over immense textual corpuses. Given an input context sequence, the model maximizes the likelihood of predicting the immediate subsequent token across billions of parameters. At inference time, these

architectures perform summarization through prompt engineering and in-context learning. By prepending a instructional prompt (such as "Summarize the following text in three concise bullet points:"), the causal decoder continues generating text, shifting from prompt consumption directly into summary generation.

While large causal language models demonstrate impressive zero-shot fluency and contextual adaptability, they present distinct operational trade-offs when deployed within dedicated document compression pipelines. Because causal decoders process source documents unidirectionally, they cannot build fully contextualized, bidirectional representations of early paragraphs informed by concluding sections. Furthermore, their immense parameter scale introduces substantial computational overhead, high GPU memory utilization, and increased inference latency compared to dedicated, task-optimized sequence-to-sequence models like BART.

5.4 Domain Adaptation and Parameter Efficiency

Deploying large pre-trained language models across specialized domains - such as legal contracts, biomedical literature, and technical white papers—frequently presents domain-shift challenges. Pre-trained checkpoints reflect the broad lexical patterns of general web and news corpora, often struggling with specialized terminology, dense domain structures, and atypical syntax when unadapted.

Domain adaptation resolves this discrepancy by exposing the pre-trained model to target-domain corpora. This can be accomplished through continued self-supervised pre-training, where the model continues its denoising or masked-language objectives on in-domain documents before task fine-tuning. This process updates the internal attention weights to recognize domain-specific jargon, technical nomenclature, and complex rhetorical patterns without catastrophic forgetting of general syntactic rules.

To reduce the computational burden of fine-tuning multi-million parameter models, Parameter-Efficient Fine-Tuning (PEFT) techniques have become a standard alternative to full-model fine-tuning. Methods such as Low-Rank Adaptation (LoRA) freeze the primary pre-trained weight matrices and inject trainable low-rank decomposition matrices into the multi-head attention projection layers. Other parameter-efficient strategies include prefix tuning, which prepends continuous, task-specific virtual token vectors to the key and value states, and adapter modules, which insert small bottleneck feed-forward layers directly after Transformer attention blocks. These techniques dramatically reduce the number of trainable parameters by up to 99%, allowing models to adapt to new linguistic complexity tiers and specialized domains while minimizing storage footprints and training latency.

Chapter 6: Generative Adversarial Networks (GANs) for Discrete Text

6.1 The Fundamental GAN Formulation (G vs D)

Generative Adversarial Networks (GANs), introduced by Goodfellow et al., frame generative modeling as a minimax zero-sum game between two competing neural networks: a Generator (G) and a Discriminator (D). In traditional continuous domains such as computer vision, the generator maps a continuous noise distribution $p_{\mathrm{z}}(\mathrm{z})$ into synthetic data samples $G(\mathrm{z})$ that emulate real data drawn from a true distribution $p_{\mathrm{data}}(\mathrm{x})$. Simultaneously, the discriminator acts as a binary classifier tasked with calculating the probability that a given sample originated from the true data distribution rather than the generative model.

The minimax optimization objective is formally defined as:

$$\min_{G}\max_{D} V(D,G) = \mathbb{E}_{\mathrm{x}\sim p_{\mathrm{data}}(\mathrm{x})}[\log D\,(\mathrm{x})] + \mathbb{E}_{\mathrm{z}\sim p_{\mathrm{z}}(\mathrm{z})}\big[\log\big(1 - D\big(G(\mathrm{z})\big)\big)\big]$$

During training, D is optimized to maximize the probability of assigning correct categorical labels to both training examples and generated samples:

Conversely, G is trained to fool the discriminator by minimizing $\log\big(1 - D\big(G(\mathrm{z})\big)\big)$, or more commonly in practical implementations, maximizing $\log D\left(G(\mathrm{z})\right)$ to circumvent early gradient saturation:

$$\nabla_{\theta_G}\frac{1}{m}\sum_{i=1}^{m}\log D\left(G\left(\mathrm{z}^{(i)}\right)\right)$$

Under theoretical equilibrium conditions, this adversarial interaction drives the generative distribution p_g to converge toward the true data distribution p_{data}, at which point the discriminator achieves an optimal non-informative classification accuracy $D(\mathrm{x}) = \frac{1}{2}$.

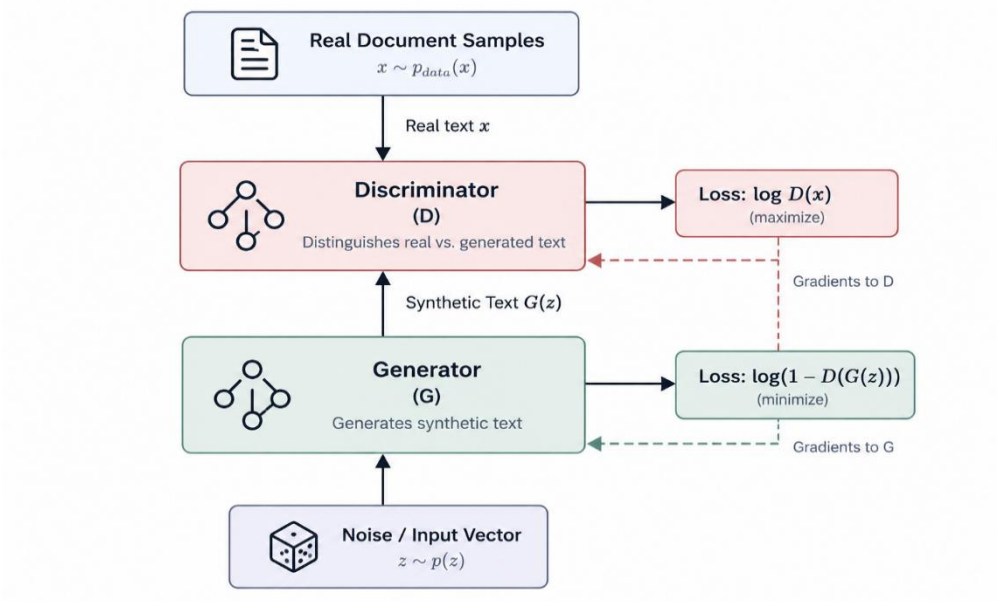


Fig 6.1: Flowchart showing architecture and workflow of GAN

6.2 The Non-Differentiability Bottleneck in Discrete Tokens

While GAN architectures achieve strong empirical success in continuous real-valued domains (e.g., image and audio synthesis), adapting the minimax framework directly to natural language processing encounters a fundamental mathematical obstacle: the discrete token selection bottleneck.

In autoregressive language generation, the generator maintains hidden representations h_t and projects them through a linear layer to emit a categorical probability distribution over the target vocabulary $\mathcal{V}$ via the Softmax function:

$$\mathrm{p}_t = \mathrm{softmax}(\mathrm{W}_v \mathrm{h}_t)$$

To emit actual text strings that can be processed by downstream layers or fed into the discriminator, the network must sample a discrete token $y_t \in \mathcal{V}$ from this probability distribution:

$$y_t = \arg\max_{w \in \mathcal{V}}(\mathrm{p}_t) \quad \text{or} \quad y_t \sim \mathrm{Categorical}(\mathrm{p}_t)$$

The arg max operation and categorical sampling steps are non-continuous step functions whose mathematical derivatives are zero almost everywhere, with infinite discontinuities at decision boundaries:

$$\frac{\partial y_t}{\partial \mathrm{p}_t} = 0 \quad \text{(almost everywhere)}$$

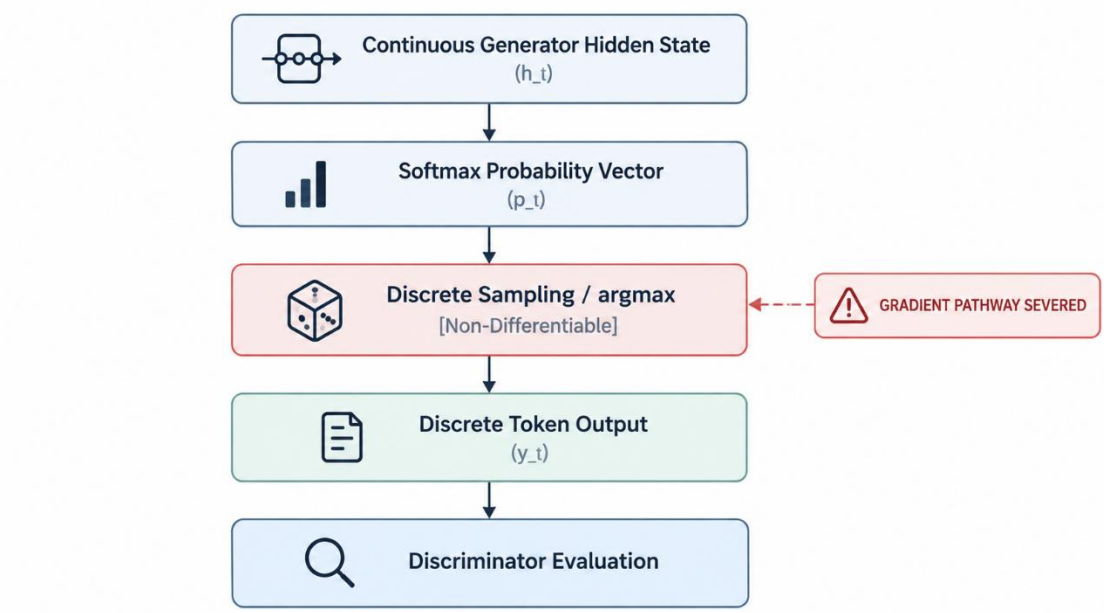


Fig 6.2: Challenges of Backpropagation in Adversarial Text Generation

Because discrete token selection severs the computational graph, backpropagation via the standard chain rule cannot calculate gradients of the discriminator's scalar output with respect to the generator's internal parameters θ_G:

$$\frac{\partial \mathcal{L}_G}{\partial \theta_G} = \frac{\partial \mathcal{L}_G}{\partial D(Y)} \cdot \frac{\partial D(Y)}{\partial Y} \cdot \underbrace{\frac{\partial Y}{\partial \mathbf{p}_t}}_{=0} \cdot \frac{\partial \mathbf{p}_t}{\partial \theta_G}$$

Consequently, small parameter updates in θ_G cannot change the sampled discrete token y_t, leaving the generator without a functional training signal to update its weights through standard gradient descent.

6.3 Reinforcement Learning Solutions: Policy Gradient & REINFORCE

6.3.1 Formulating Text Generation as an MDP

In this formulation:

a) *Agent*: The generative language model (G).
b) *State* (s_t): The sequence of tokens generated up to time step t, denoted as $Y_{1:t-1} = (y_1, y_2, \dots, y_{t-1})$, conditioned on the source text **x**.
c) *Action* (a_t): The selection of the next discrete token y_t chosen from the vocabulary action space $\mathcal{V}$.
d) *Policy* ($\pi_\theta(a_t \mid s_t)$): The conditional probability distribution over the vocabulary emitted by the generator:

$$\pi_\theta(y_t \mid Y_{1:t-1}, \mathbf{x}) = P(y_t \mid Y_{1:t-1}, \mathbf{x}; \theta)$$

e) *Reward* (R): A scalar evaluation signal provided by the discriminator D_ϕ only after the complete sequence $Y_{1:T}$ is generated

6.3.2 The Policy Gradient Theorem and REINFORCE

The goal of the generator is to maximize the expected cumulative reward of the generated sequences under its parameterized policy:

$$J(\theta) = \mathbb{E}_{Y_{1:T} \sim \pi_\theta}[R(Y_{1:T})] = \sum_{Y_{1:T}} P\,(Y_{1:T}; \theta) R(Y_{1:T})$$

Applying the log-derivative trick ($\nabla_\theta P = P \nabla_\theta \log P$), the gradient of the expected reward with respect to the generator parameters θ is derived via the Policy Gradient Theorem:

$$\nabla_\theta J(\theta) = \mathbb{E}_{Y_{1:T} \sim \pi_\theta}\left[\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta\,(y_t \mid Y_{1:t-1}) Q(s_t, y_t)\right]$$

Using the REINFORCE algorithm (Williams, 1992), this expected value is approximated empirically using Monte Carlo sampling over K generated rollouts:

$$\nabla_\theta J(\theta) \approx \frac{1}{K} \sum_{k=1}^{K} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta \left(y_t^{(k)} \mid Y_{1:t-1}^{(k)}\right) \left(R\left(Y_{1:T}^{(k)}\right) - b(s_t)\right)$$

Where $b(s_t)$ represents a baseline state-value estimate subtracted from the reward to reduce the variance of the gradient estimator without introducing bias.

Component	Standard Supervised Seq2Seq	Policy Gradient RL Formulation
Loss Signal	Token-level Cross-Entropy ($-\log P\,(y_t^*)$)	Sequence-level Discriminator Reward ($R(Y_{1:T})$)
Input Supervision	Ground-truth reference tokens	Autoregressively sampled policy rollouts
Exposure Bias	High (Teacher forcing discrepancy)	Low (Trained directly on self-generated paths)
Gradient Flow	Direct backpropagation through Softmax	Expected score-function trick ($\nabla_\theta \log \pi_\theta$)

Table 6.1: Comparison between Standard Supervised Seq2Seq and Policy Gradient RL Formulation

6.4 Actor-Critic Architectures for Natural Language

While the basic REINFORCE algorithm circumvents the non-differentiability bottleneck, it exhibits high gradient variance. Because a reward $R(Y_{1:T})$ is returned only after the complete sequence is synthesized, every token generated in the sequence receives the same global terminal reward, regardless of whether a specific word choice improved or impaired the overall summary quality.

To address this credit assignment challenge, natural language frameworks incorporate Actor-Critic architectures:

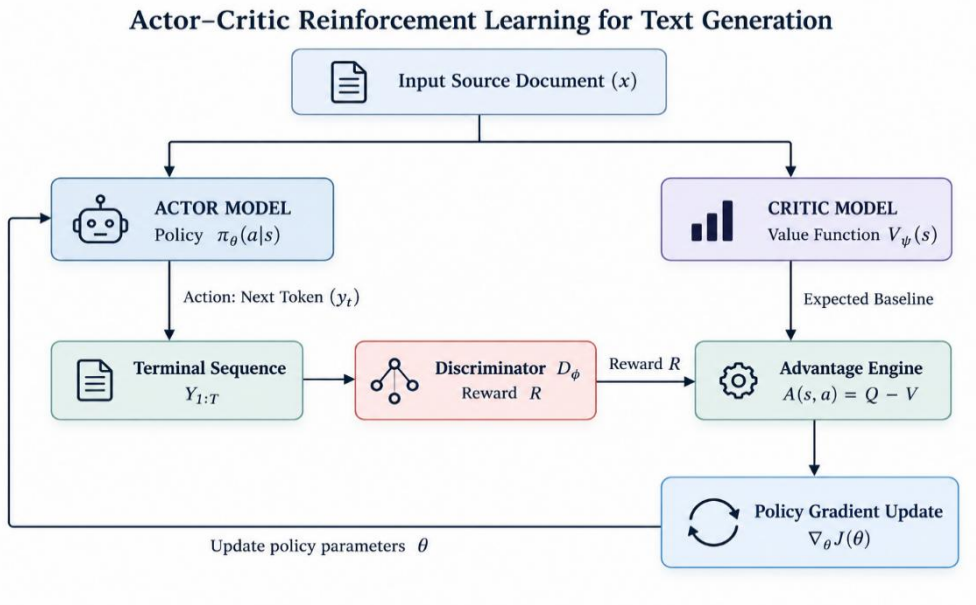


Fig 6.3: Actor-Critic Architecture Workflow Diagram

6.4.1 The Actor Network (Policy Formulation)

The **Actor** is the sequence-to-sequence generator G_θ. It maintains the generative policy $\pi_\theta(y_t \mid Y_{1:t-1})$ and is responsible for selecting discrete actions (tokens) from the vocabulary.

6.4.2 The Critic Network (Value Estimation)

The "Critic" is an auxiliary neural network $V_\psi(s_t)$ parameterized by ψ. It evaluates intermediate generation states, predicting the expected cumulative reward from the current partial sequence $Y_{1:t}$ until sequence completion:

$$V_\psi(s_t) \approx \mathbb{E}[R(Y_{1:T}) \mid Y_{1:t}]$$

6.4.3 Advantage Function and Temporal Difference Updates

Instead of weighting policy updates with raw terminal rewards, the Actor is updated using the **Advantage Function** $A(s_t, y_t)$, which measures how much better a chosen token is compared to the expected baseline value of that state:

$$A(s_t, y_t) = Q(s_t, y_t) - V_\psi(s_t)$$

Using a 1-step Temporal Difference (TD) target, the intermediate action-value $Q(s_t, y_t)$ is approximated as:

$$Q(s_t, y_t) \approx r_t + \gamma V_\psi(s_{t+1})$$

The Actor and Critic parameters are updated through alternating optimization:

- **Critic Update:** Minimize the Mean Squared Error (MSE) between the predicted state values and the observed TD targets:

$$\mathcal{L}(\psi) = \frac{1}{T}\sum_{t=1}^{T}\left(r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t)\right)^2$$

- **Actor Update:** Scale the log-probability of the policy by the calculated advantage estimate:

$$\nabla_\theta \mathcal{L}(\theta) = \frac{1}{T}\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta\,(y_t \mid s_t) A(s_t, y_t)$$

By balancing sequence-level evaluation with fine-grained step-wise baseline estimation, Actor-Critic architectures provide stable gradient updates, resolve the discrete credit assignment problem, and guide text generation models toward high factual consistency and target complexity standards.

Chapter 7: The Reado.ai Tri-Level Contextual Text GAN

7.1 System Architecture of the Reado.ai Engine

The Reado.ai engine is engineered to resolve the fundamental trade-off between semantic retention, structural complexity, and reader cognitive budgets. Instead of generating a single, unyielding output, the platform formulates text summarization as a conditioned generative process. Given a source passage $\mathrm{x} = (x_1, x_2, \ldots, x_M)$ and an explicit user-selected complexity tier $c \in \{\text{Easy,Hard,Editorial}\}$, the engine samples an output sequence $\mathrm{y} = (y_1, y_2, \ldots, y_N)$ according to a conditioned parametric policy:

$$P(\mathrm{y} \mid \mathrm{x}, c; \theta) = \prod_{t=1}^{N} P\left(y_t \mid y_{<t}, \mathrm{x}, c; \theta\right)$$

The system integrates a pre-trained sequence-to-sequence Transformer backbone (facebook/bart-large-cnn) with an adversarial discriminator conditioned on discrete reading targets. The generator is guided by a hybrid objective that combines token-level supervised cross-entropy with sequence-level policy gradient rewards emitted by the complexity-aware discriminator.

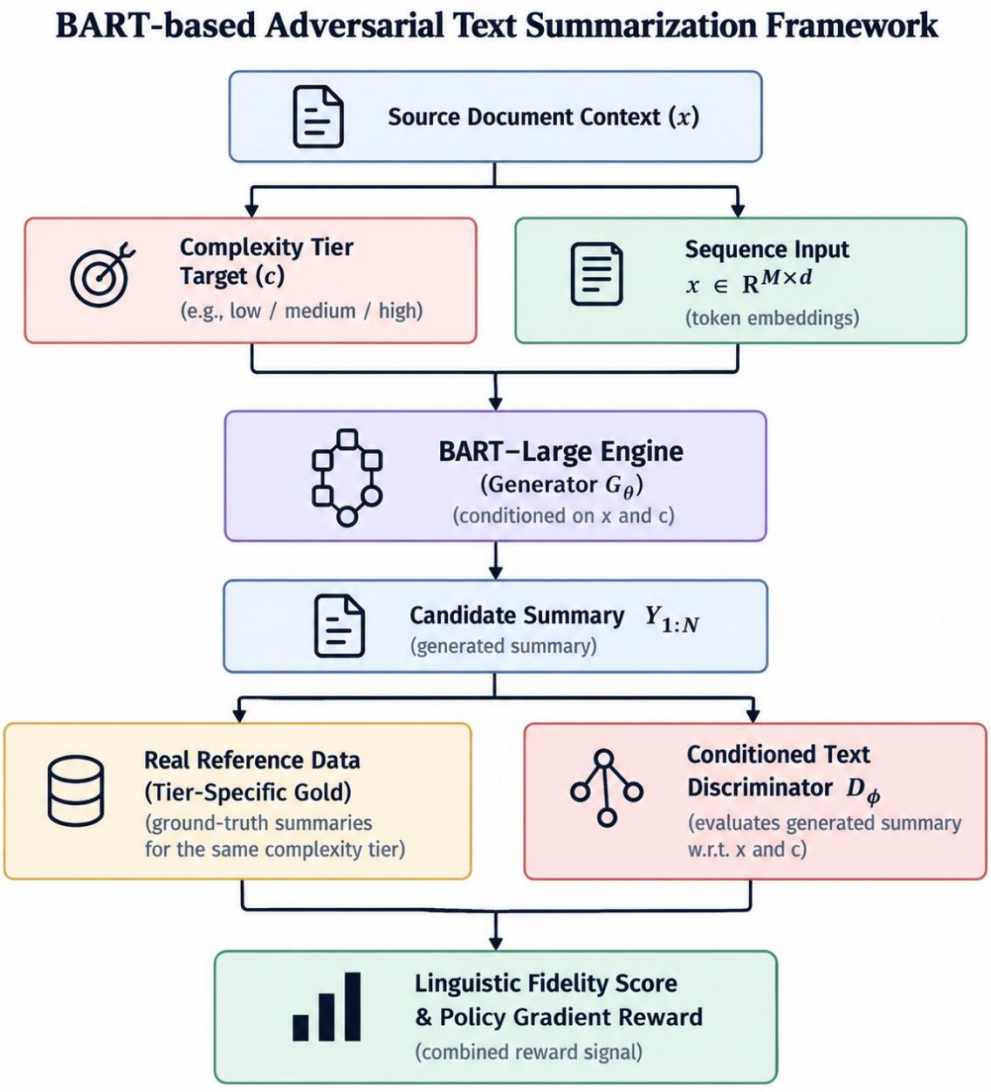


Fig 7.1: Complexity-Aware Adversarial Summarization Pipeline

7.2 Level 1: Easy Mode (Syntactic Simplification)

Easy Mode is calibrated for high-speed information acquisition, mobile interfaces, and readers seeking quick executive takeaways. The operational goal is maximum semantic compression (85% to 92% word reduction relative to the source) without omitting primary subject-verb-object propositions.

To enforce syntactic simplicity, the decoding configuration enforces three operational constraints:

- *High Length Penalty* **($\alpha = 2.2$):** Penalizes long candidate sequences during beam search decoding, encouraging the model to prune non-essential discourse and terminate generation once core propositions are stated.
- *Tight Token Ceilings*: Constrains the minimum and maximum token windows to $25 \leq N \leq 55$ tokens, preventing conversational rambling or anecdotal expansion.
- *Syntactic Pruning*: Suppresses multi-clause nestings, relative clauses, and parenthetical interruptions, producing concise declarative sentences that maximize readability scores.

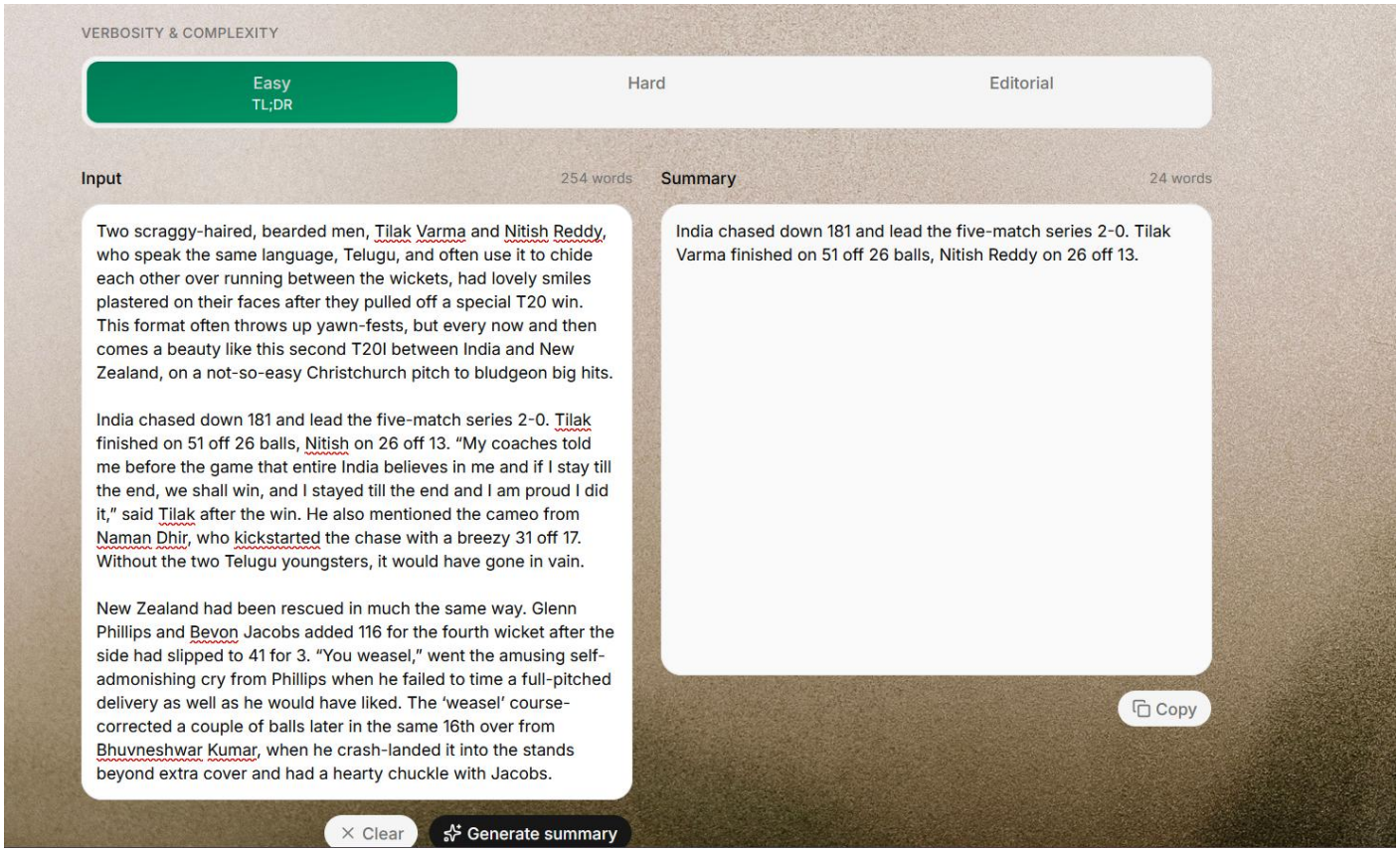


Fig 7.2: Easy Mode Complexity demonstration on live UI

7.3 Level 2: Hard Mode (Analytical and Structured)

Hard Mode is formulated for technical researchers, intelligence analysts, and domain specialists who require structured informational density. Rather than prioritizing extreme lexical brevity, this tier preserves quantitative data points, causal assertions, and intermediate contextual dynamics.

The decoding mechanics are configured to balance compression with evidentiary depth:

- Neutral Length Penalty ($\alpha = 1.0$): Maintains a balanced trade-off between sequence brevity and exhaustive token likelihood, allowing compound multi-sentence constructions to complete naturally.
- Intermediate Token Allocation: Bounded between $75 \leq N \leq 130$ tokens, ensuring sufficient representational capacity to retain secondary actors and comparative metrics.
- Logical Dependency Preservation: Preserves relational connective tokens ("after," "while," "consequently") to maintain the exact causal relationships established in the source document.

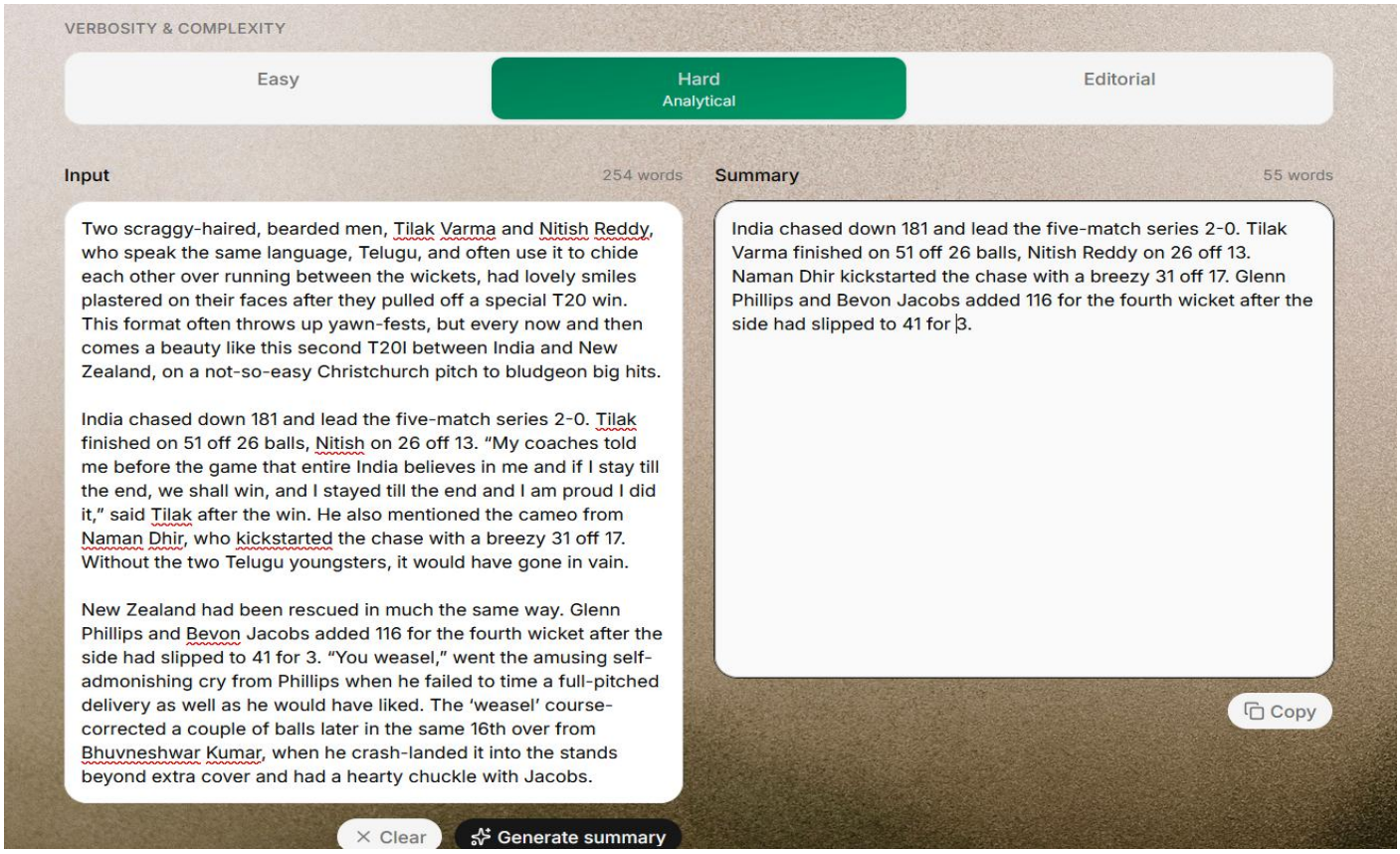


Fig 7.3: Easy Mode Complexity demonstration on live UI

7.4 Level 3: Editorial Mode (Publication-Grade Synthesis)

Editorial Mode is calibrated for newsroom publishing, professional briefings, and long-form analytical digests. The objective is to synthesize narrative flow, preserve rhetorical style, and retain memorable quotations while achieving moderate semantic compression (50% to 60%).

The operational rules for this profile prioritize discourse depth:

- Low Length Penalty ($\alpha = 0.6$): Explicitly rewards longer hypotheses, giving the beam search decoder headroom to construct compound-complex sentences and elaborate on narrative subplots.
- Expanded Output Boundaries: Bounded between $160 \leq N \leq 260$ tokens, providing the space needed for full contextual development.
- Direct Discourse and Rhetorical Retention: Retains direct quotes, qualitative adjectives, and transition markers, yielding publication-grade journalistic prose that requires zero post-editing.

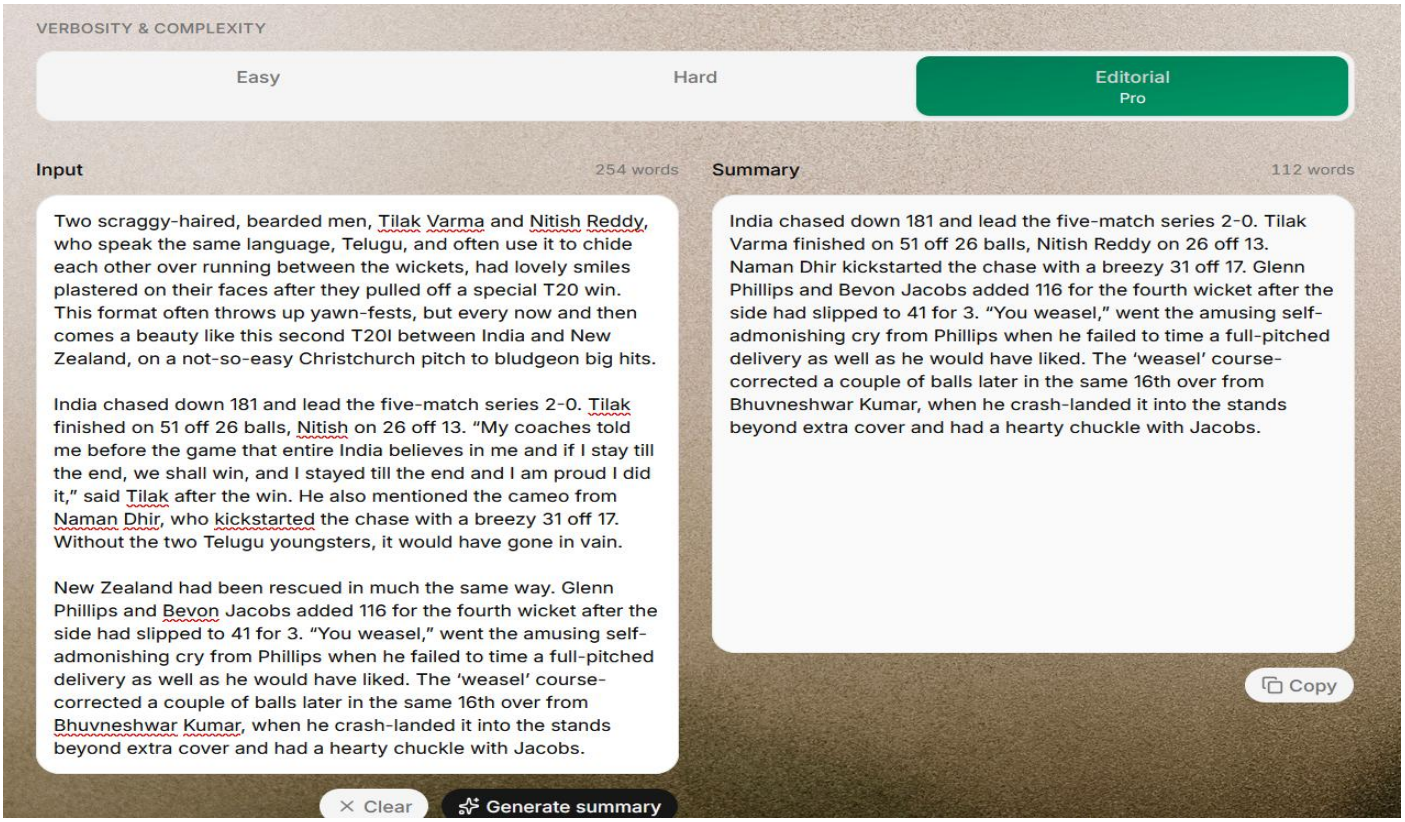


Fig 7.4: Editorial Mode Complexity demonstration on live UI

Parameter / Metric	**Level 1: Easy (TL;DR)**	**Level 2: Hard (Analytical)**	**Level 3: Editorial (Pro)**
Target Audience	Mobile users, quick briefing	Researchers, data analysts	Journalists, formal reviews
Token Budget ($[min, max]$)	[25,55] tokens	[75,130] tokens	[160,260] tokens
Length Penalty (α)	2.2 (Aggressive brevity)	1.0 (Neutral likelihood)	0.6 (Length incentivized)
Compression Ratio	$85\% - 92\%$ reduction	$70\% - 80\%$ reduction	$50\% - 60\%$ reduction
Syntactic Profile	Short declarative clauses	Compound analytical syntax	Stylistic complex sentences

Parameter / Metric	Level 1: Easy (TL;DR)	Level 2: Hard (Analytical)	Level 3: Editorial (Pro)
Empirical Output Word Count	24 words	55 words	112 words

Table 7.1: Summary Table for 3-Tier Verbosity Breakdowns

7.5 Conditioned Discriminator Formulation

To train the generator across discrete reading tiers, the adversarial framework utilizes a Conditioned Discriminator (D_ϕ). In standard GAN formulations, the discriminator evaluates only whether an output is "real" or "machine-generated." In Reado.ai, the discriminator is augmented with a condition embedding e_c mapping to the target complexity level c.

The conditioned discriminator must simultaneously solve two classification objectives:

1. Adversarial Realism: Distinguish whether sequence y is a natural, coherent human-authored text or a synthetic candidate emitted by the generator.
2. Tier Compliance: Verify whether sequence y satisfies the specific stylistic, syntactic, and length constraints demanded by condition c.

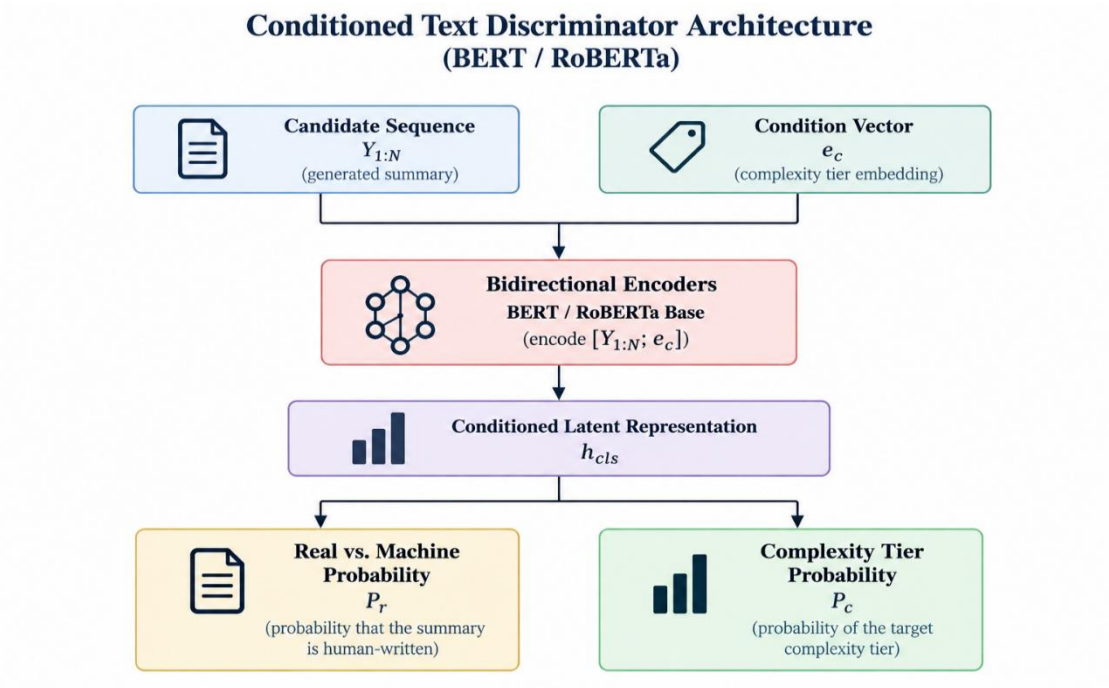


Fig 7.5: Conditional Representation Learning for Summary Quality and Complexity Assessment

Chapter 8: Controlling Hallucination and Repetition in Text GANs

8.1 Hallucination Modes in Deep Generative Models

In neural abstractive summarization, hallucination refers to generated content that is nonsensical or unfaithful to the source document. While abstractive systems are explicitly trained to paraphrase and condense, standard maximum likelihood estimation (MLE) over large corpora encourages models to rely heavily on language modeling priors rather than strict source-text conditioning.

In sequence-to-sequence summarization, hallucinations are broadly categorized into two operational modes:

- *Intrinsic Hallucinations*: The generated text directly contradicts assertions made in the source text. For example, if a source passage states, *"The company reported a quarterly loss of $12 million due to supply disruptions,"* an intrinsic hallucination might state, *"The company generated $12 million in net profit."* This mode alters quantities, inverts causal relationships, or misattributes actions between distinct entities.
- *Extrinsic Hallucinations*: The generated text introduces facts, entities, or claims that cannot be grounded in or verified by the source text. While these external claims may be factually accurate in the real world—derived from world knowledge encoded during large-scale pre-training—they represent ungrounded interpolations within the specific document summarization task.

Intrinsic Hallucination	**Extrinsic Hallucination**
Contradicts source facts	Introduces external claims
Entity misattribution	Ungrounded world knowledge
Inverts causal dependencies	Plausible but unverifiable

Table 8.1: Divisions of Source Text Context

8.2 Degenerative Repetition & Circular Loops

A persistent failure mode in autoregressive decoding is degenerative repetition, where a model becomes trapped in circular lexical loops. This behavior typically emerges during beam search decoding when handling complex multi-clause contexts or tight compression constraints.

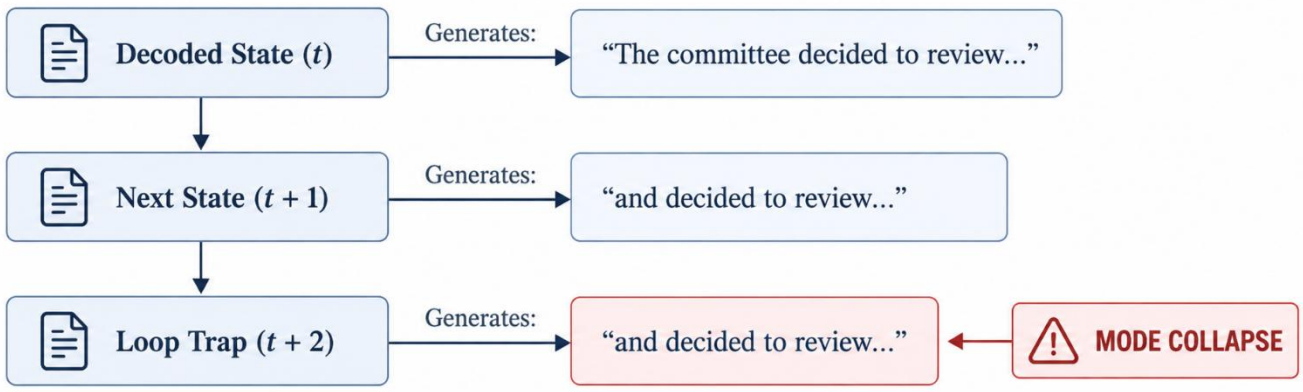


Fig 8.1: Degeneration and Repetition Loops in Text Generation

This pathology stems from two structural factors:

1. *Exposure Bias*: Models are trained using teacher forcing, where ground-truth tokens are provided at every step. At test time, the model conditions on its own previously sampled tokens. An improbable or slightly skewed token emission shifts the hidden state into out-of-distribution regions, where repetitive n-grams receive artificially elevated probabilities.
2. *High-Probability Attractor States*: In beam search, common phrases (e.g., *"in addition to the fact that"*) carry high marginal probabilities. Once emitted, the causal decoder assigns high conditional probabilities to repeating that exact transition, creating an attractor state that suppresses terminal tokens.

8.3 N-Gram Blocking and Coverage Penalties

To eliminate degenerative loops and ensure balanced document coverage, the decoding pipeline integrates two deterministic constraints: n-gram blocking and coverage penalties.

8.3.1 Dynamic N-Gram Blocking

N-gram blocking enforces a hard constraint during beam search expansion. If a candidate token w would complete an n-gram that has already been emitted within the current hypothesis beam, its predictive probability is forced to zero prior to softmax normalization:

$$P(y_t = w \mid y_{<t}, \mathrm{x}) = 0 \quad \text{if } (y_{t-n+1}, \ldots, y_{t-1}, w) \in \text{Seen-N-Grams}(y_{<t})$$

In the Reado.ai inference engine, setting no_repeat_ngram_size = 3 eliminates circular tri-gram loops without disrupting valid lexical pairings.

8.3.2 Attention Coverage Regularization

To prevent over-attending to specific source sentences while ignoring others, a coverage vector c^t accumulates attention distributions over previous decoding steps:

$$\mathrm{c}^t = \sum_{t\prime=1}^{t-1} \mathrm{a}^{t\prime}$$

A coverage loss penalty is applied to minimize overlap between current attention weights a^t and the historical coverage vector c^t:

$$\mathcal{L}_{\text{cov}}^t = \sum_i \min(a_i^t, c_i^t)$$

This penalty discourages the attention heads from repeatedly focusing on the same source tokens, ensuring that decoding advances through the narrative progression of the document.

8.4 Discriminator Reward Regularization

In an adversarial text generation framework, an unregularized discriminator can exploit surface-level artifacts, providing noisy or unstable reward signals that destabilize the generator. To maintain steady reinforcement learning updates, Reado.ai applies two regularizations to the discriminator reward function:

Regularization Strategy	**Operational Role**	**Algorithmic Impact**
Entropy Regularization	Penalizes overconfident policy distributions	Prevents premature convergence to repetitive phrasing
Factual Consistency Reward	Evaluates entity overlap against source context	Penalizes intrinsic hallucinations
Reward Clipping	Restricts reward outputs to $[-R_{\max}, R_{\max}]$	Prevents extreme gradients from destabilizing the generator

Table 8.2: Brief Description of Regularization strategy adopted

The total sequence-level reward $R_{\text{total}}(\mathbf{y})$ balances adversarial realism with factual fidelity:

$$R_{\text{total}}(\mathbf{y}) = D_\phi(\mathbf{y}, c) + \beta \cdot \text{Score}_{\text{Fact}}(\mathbf{y}, \mathbf{x}) - \lambda \cdot \mathcal{L}_{\text{rep}}(\mathbf{y})$$

Chapter 9: Benchmark Datasets and Data Collection

9.1 Benchmark Corpora: CNN/DailyMail and XSum

The empirical training, validation, and benchmarking of abstractive summarization frameworks require large, standardized textual corpora characterized by high editorial quality, broad thematic coverage, and verified document-summary alignments. The development and calibration of the Reado.ai pipeline utilize two foundational NLP benchmark datasets: CNN/DailyMail and Extreme Summarization (XSum).

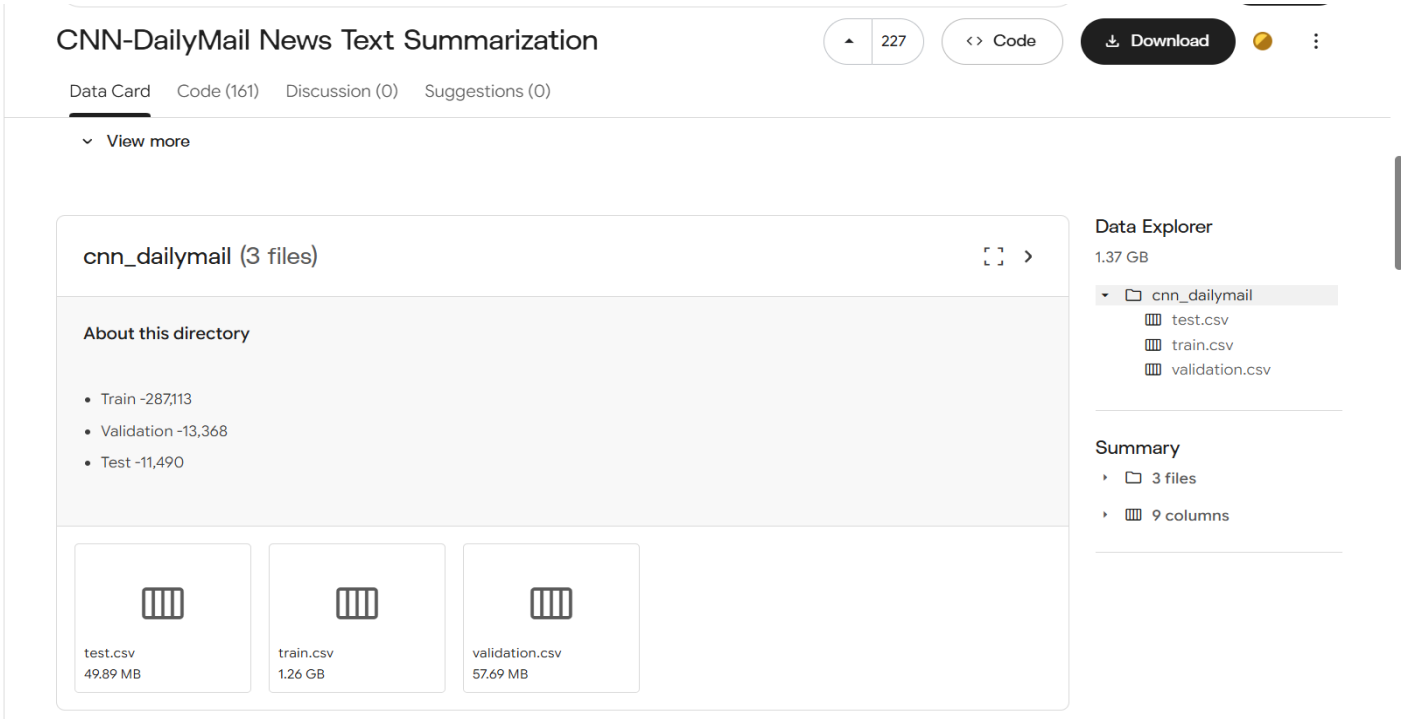


Fig 9.1: CNN/DailyMail Dataset presence on Kaggle.com

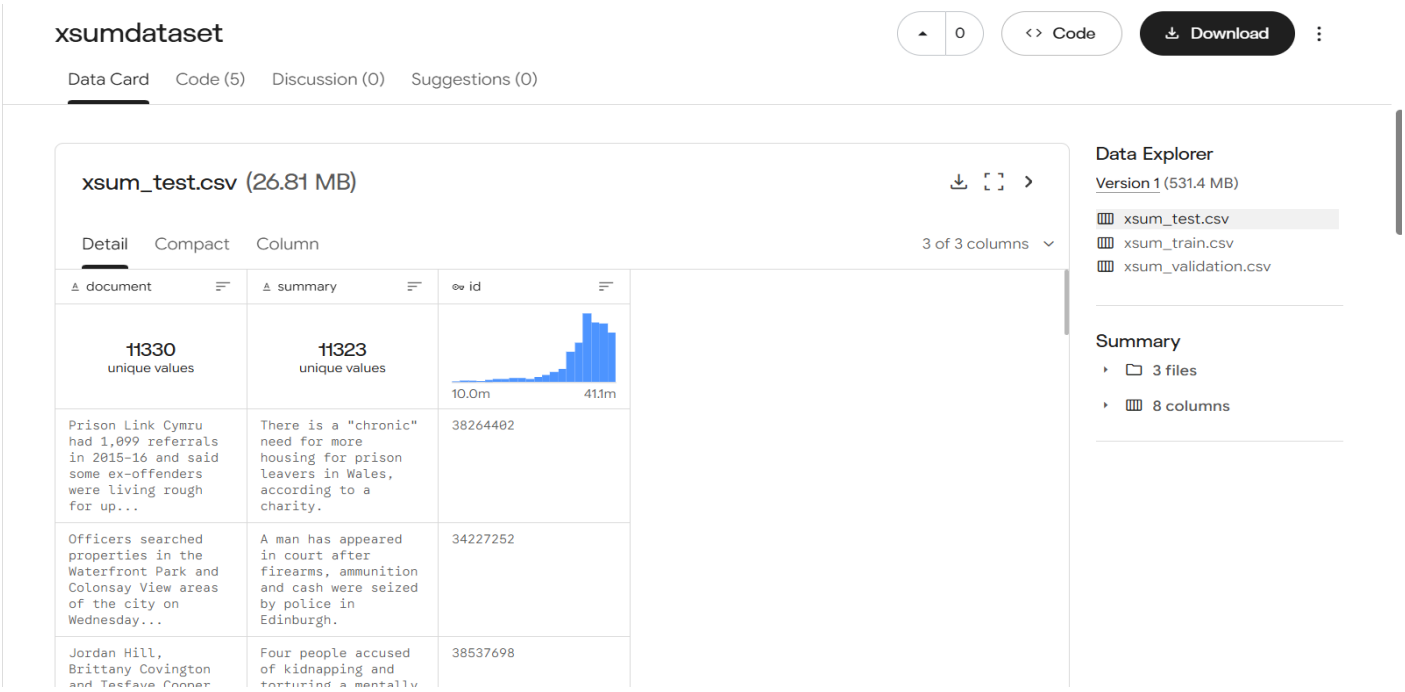


Fig 9.2: XSum Dataset presence on Kaggle.com

9.1.1 The CNN/DailyMail Benchmark

Originally compiled by Hermann et al. for question-answering and modified by Nallapati et al. for abstractive summarization, the CNN/DailyMail dataset contains approximately 312,000 paired examples of news articles accompanied by human-authored bullet-point highlights. The source articles average approximately 780 words, whereas the reference summaries average 3 to 4 sentences (approximately 50 to 65 words).

Because the target bullet points frequently borrow verbatim clauses from the primary narrative, CNN/DailyMail exhibits moderate extractive density. This characteristic makes it suitable for fine-tuning baseline models to extract salient propositions, retain named entities, and assemble cohesive analytical summaries, aligning with the operational objectives of Reado.ai's Level 2 (Hard Mode).

9.1.2 The Extreme Summarization (XSum) Benchmark

Constructed by Narayan et al., the XSum dataset comprises 226,711 British Broadcasting Corporation (BBC) articles spanning diverse reporting categories, including politics, public policy, technology, and international relations. Each article is paired with a professionally written, single-sentence summary (averaging 23 words) derived from the introductory lead sentence of the original publication.

Unlike CNN/DailyMail, the reference summaries in XSum exhibit low extractive overlap and high novel n-gram density. Generating an XSum summary requires the model to synthesize information across disparate clauses into an overarching proposition. This characteristic makes XSum an effective training baseline for high-compression, low-word-count targets, informing the syntactic simplification rules of Reado.ai's Level 1 (Easy Mode).

9.2 Custom Editorial Corpus Curation

While CNN/DailyMail and XSum provide baselines for general and extreme compression, neither dataset provides the stylistic range, narrative flow, and analytical framing needed for Level 3 (Editorial Mode). To support this advanced tier, a targeted editorial corpus was curated.

The curation pipeline was designed to gather long-form analytical journalism and editorial commentary across multiple domains:

a) *Source Selection*: Articles were collected from syndicated editorial columns, analytical reviews, and long-form investigative journalism covering current affairs, economic analyses, and technological trends.
b) *Text Preprocessing and Boilerplate Removal*: Raw web scrapings were stripped of HTML artifacts, social media embeds, author bios, boilerplate disclaimers, and wire stamps (e.g., "CHRISTCHURCH (Reuters)").

c) *Lexical and Complexity Filtering*: Source texts were filtered based on length (retaining articles between 800 and 2,000 words) and syntactic richness (retaining documents with Flesch Reading Ease scores below 50, indicating advanced prose).

d) *Reference Synthesis*: High-grade editorial summaries (spanning 110 to 180 words) were aligned with the filtered documents, emphasizing direct discourse retention, contextual nuance, and narrative flow.

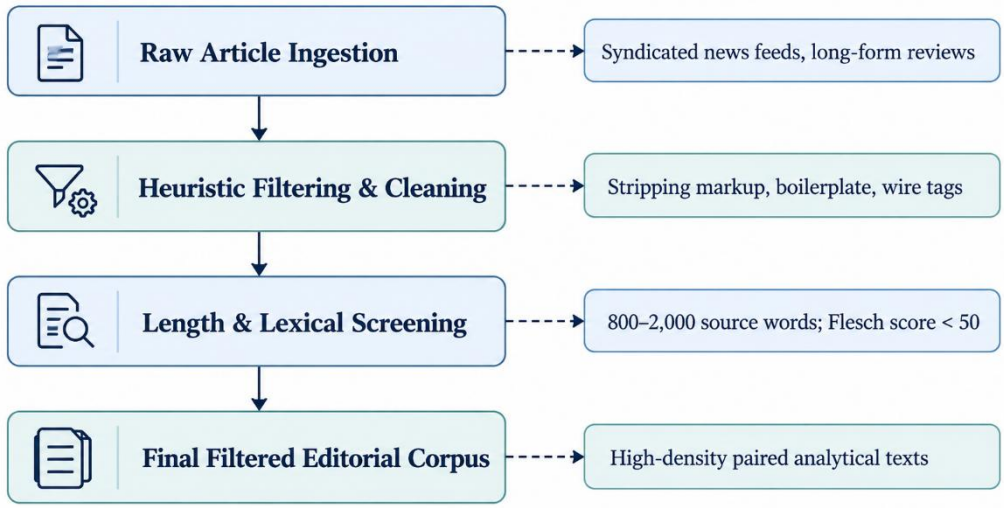


Fig 9.3: Data Collection, Filtering, and Corpus Construction for Text Summarization

9.3 Lead Bias and Document Length Distributions

A prominent challenge in processing news-based corpora is lead bias (often called the inverted pyramid writing style). In journalistic writing, editors and reporters traditionally concentrate the primary factual assertions—the *who*, *what*, *where*, and *when*—within the first one or two paragraphs, using subsequent sections for historical context, secondary quotations, and supplementary anecdotes.

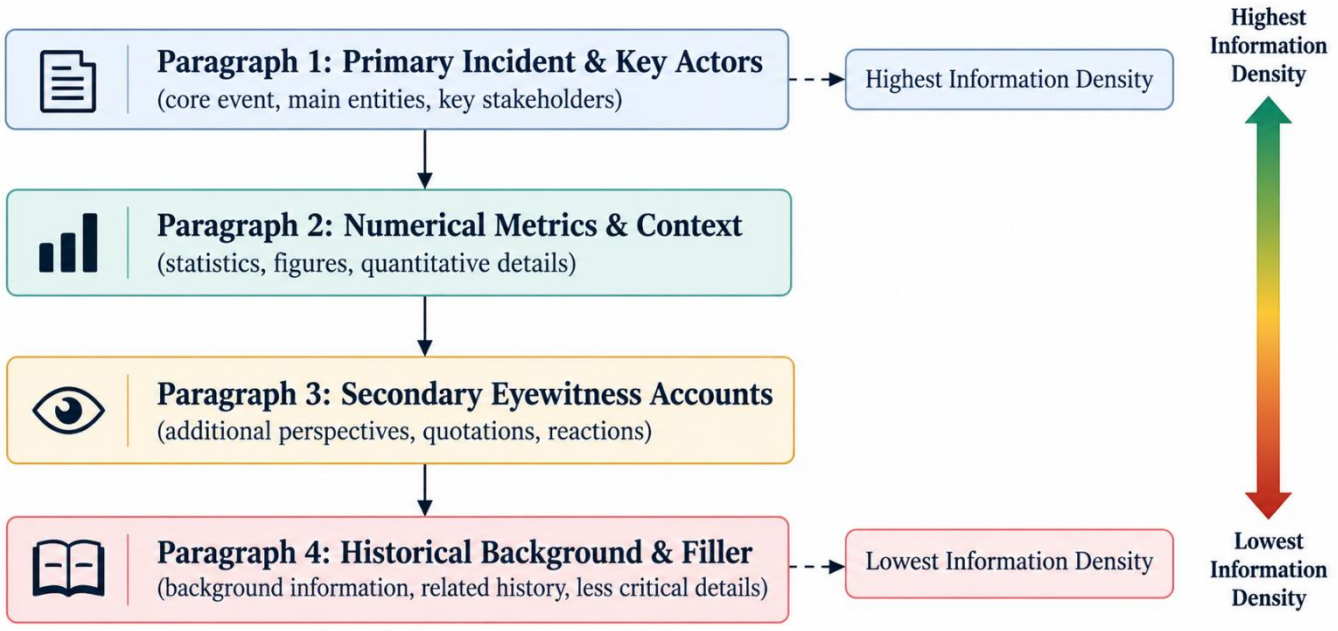


Fig 9.4: Information Density Hierarchy Flowchart

When neural models are trained uncritically on news corpora, they learn to exploit lead bias as an optimization shortcut: the network simply attends to the first three sentences of the input document and ignores subsequent text. This shortcut causes models to fail on non-news literature, academic papers, and technical reports where core findings appear in concluding sections.

To mitigate lead bias, the training pipeline balances document lengths and applies truncation boundaries:

Corpus Metric	**CNN/DailyMail**	**XSum**	**Curated Editorial Corpus**
Mean Source Word Count	781 words	431 words	1,120 words
Mean Summary Word Count	56 words	23 words	128 words
Average Compression Ratio	92.8%	94.6%	88.5%
Extractive Fragment Density	High (≈ 0.72)	Low (≈ 0.28)	Moderate (≈ 0.45)
Input Truncation Boundary	1,024 subwords	512 subwords	1,024 subwords

Table 9.1: Metrics Evaluations and Description of CNN/Daily Mail and Xsum Datasets

By capping input contexts at 1,024 subword tokens, the pipeline captures both the introductory leads and the supporting analytical clauses distributed across the middle portions of documents.

9.4 Train, Validation and Test Splits

To ensure reproducible empirical evaluation and prevent data contamination across model iterations, all datasets were partitioned into non-overlapping splits. Standard partitioning conventions were preserved for benchmark sets to enable direct comparisons with established literature.

Split Type	**Allocation (%)**	**Primary Operational Objectives**
Training Split	80%	• Model backpropagation and gradient updates • Adversarial discriminator reward optimization • Checkpoint parameter convergence

Split Type	Allocation (%)	Primary Operational Objectives
Validation Split	10%	• Hyperparameter calibration and threshold tuning • Epoch-level loss monitoring • Early stopping trigger to prevent overfitting
Test Split	10%	• Final unseen ROUGE-1, ROUGE-2, and ROUGE-L evaluation. • Readability metric benchmarking (Flesch-Kincaid). • Qualitative error analysis and inference latency testing.

Table 9.2: Data Partitioning and Splitting View

The unified training and evaluation environment divides samples as follows:

- *Training Set (80%):* Utilized for parameter optimization, backpropagation updates, and policy gradient reward learning. The model updates its encoder-decoder and discriminator weights over this split.
- *Validation Set (10%):* Evaluated periodically at the conclusion of each training epoch to track validation loss, compute intermediate ROUGE scores, guide learning rate schedules, and trigger early stopping to prevent overfitting.
- *Test Set (10%):* Held out strictly until final model evaluation. No hyperparameters, decoding thresholds, or length penalties are tuned against this split. Final quantitative metrics—including ROUGE scores, Flesch-Kincaid grade levels, and inference latency benchmarks—are derived entirely from these unseen samples.

Chapter 10: Data Preprocessing and Tokenization Pipelines

10.1 Subword Tokenization: BPE vs SentencePiece

Modern deep neural language models process text by decomposing raw character sequences into subword units. Traditional word-level tokenization assigns an isolated integer index to every distinct word in a language, which generates an unwieldy vocabulary matrix and inevitably fails when encountering out-of-vocabulary (OOV) tokens, morphological variations, or typographical misspellings. Character-level tokenization eliminates OOV errors entirely, but produces excessively long sequence representations that strain Transformer self-attention layers with quadratic computational costs. Subword tokenization balances this trade-off by breaking frequent words into full tokens while decomposing rare or complex words into frequent subword fragments.

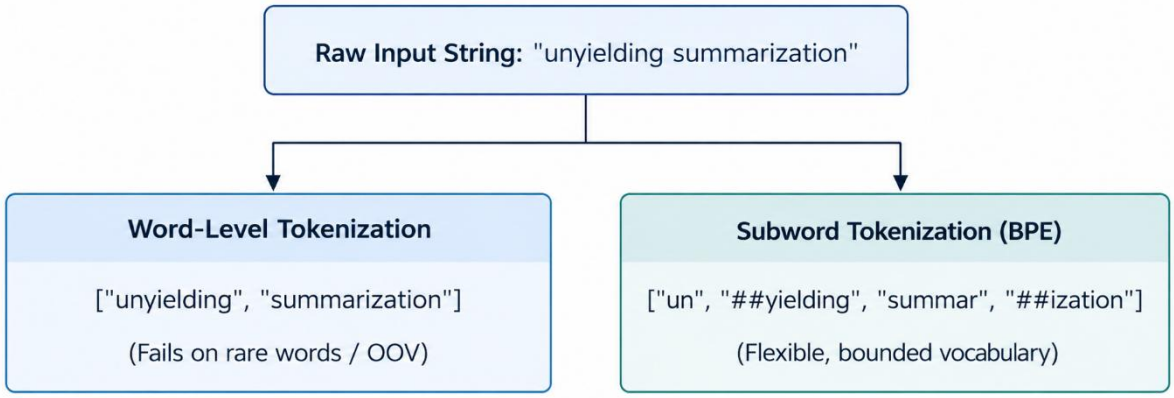


Fig 10.1: Process showing working of Tokenization

The two dominant subword tokenization frameworks in abstractive text generation are **Byte-Pair Encoding (BPE)** and **SentencePiece**:

- *Byte-Pair Encoding (BPE):* Formulated for data compression by Gage and adapted for NLP by Sennrich et al., BPE initializes its vocabulary with all individual base characters present in a training corpus. It iteratively counts all co-occurring symbol pairs and merges the most frequent adjacent pair into a single new subword token. This merge operation repeats for a pre-defined number of iterations until the target vocabulary size is reached. In the BART architecture, Byte-Level BPE processes text at the byte level rather than relying on standard Unicode whitespace splits, allowing it to tokenize any arbitrary Unicode string without requiring an explicit unknown (<unk>) token.
- *SentencePiece*: Developed by Kudo and Richardson, SentencePiece treats input text as a continuous raw byte or character sequence without assuming whitespace acts as an explicit word delimiter. Whitespace characters are converted into an explicit meta-symbol (such as _), after which either BPE or Unigram language modeling algorithms segment the sequence. SentencePiece is widely adopted in

language models such as T5, providing language-agnostic tokenization across both spaced and unspaced scripts.

Tokenization Metric	Byte-Level BPE (Used in BART)	SentencePiece (Used in T5)
Whitespace Handling	Pre-tokenizes via regex boundaries	Treats whitespace as an explicit character (_)
Vocabulary Initialization	256 basic byte tokens	Unique characters extracted from corpus
OOV Handling	Complete absence of OOV (maps to bytes)	Low OOV rate via Unigram fallback
Vocabulary Scale	50,265 subwords	32,128 subwords
Primary System Use	Reado.ai Backbone (bart-large-cnn)	Text-to-Text Transfer Framework

Table 10.1: Tokenization Mertics

10.2 Vocabulary Truncation & Special Token Injection

To maintain bounded tensor dimensions during matrix multiplications, subword vocabularies are constrained to a fixed upper limit. The vocabulary of the facebook/bart-large-cnn backbone utilized in Reado.ai is bounded at 50,265 tokens. Token IDs beyond this range are pruned, balancing memory consumption within the embedding matrix against token fragmentation rates.

Beyond semantic lexical units, sequence-to-sequence Transformers rely on structural control tokens to demarcate context boundaries, regulate generation stops, and align sequence states:

a) *Beginning-of-Sequence* (<s>): Prepended to the encoder input sequence to signal the start of a document context.
b) *End-of-Sequence* (</s>): Appended to denote the termination of the source sequence. During decoder inference, emitting this token signals that generation is complete, halting the autoregressive beam search.
c) *Padding Token* (<pad>): Inserted into batched sequence arrays to equalize length variations, ensuring that all sample tensors within a batch share uniform spatial dimensions.

d) Mask Token (<mask>): Replaces damaged spans during self-supervised denoising pre-training, teaching the model to infer missing semantic context.

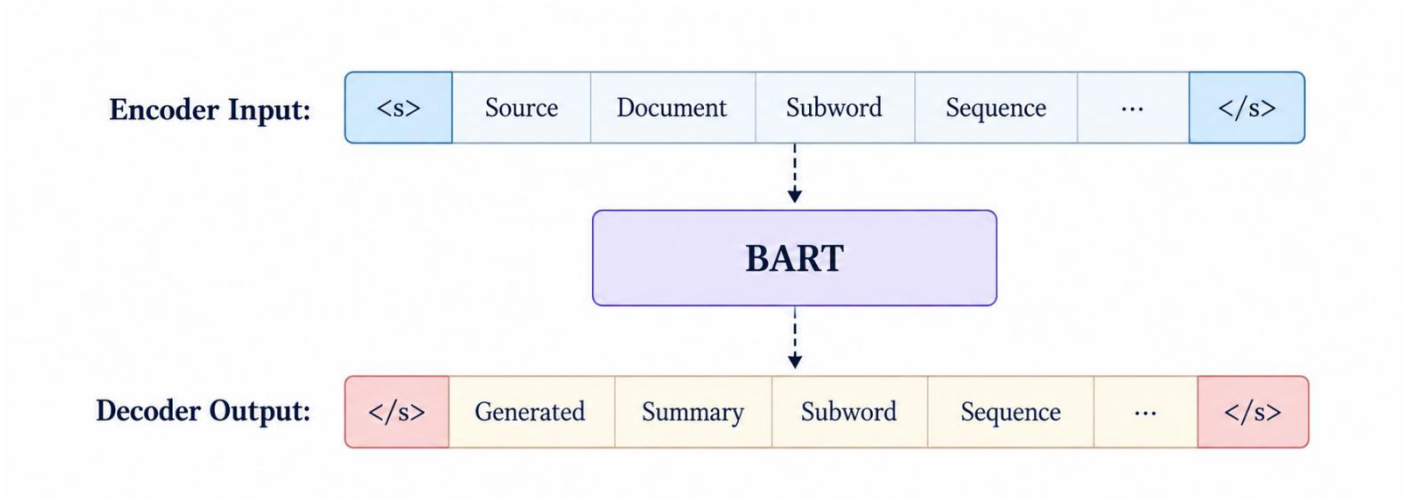


Fig 10.2: BART Input–Output Sequence Representation

10.3 Dynamic Sequence Padding and Collators

In deep neural network training, samples are grouped into multi-sequence batches to maximize tensor throughput on parallel GPU architectures. Because source documents and target summaries vary in length, sequences within a batch must be aligned to uniform token counts.

A naive approach is static padding, where every sequence in the dataset is padded to the maximum possible sequence window (e.g., 1,024 tokens). However, this introduces substantial computational inefficiency: if a batch contains articles averaging only 400 tokens, more than half of the processed tensor operations are spent on non-informative <pad> tokens, squandering memory bandwidth and computational cycles.

To resolve this inefficiency, Reado.ai utilizes **Dynamic Sequence Padding** via customized Data Collators:

1. *Batch-Wise Length Calculation*: The collator inspects only the samples allocated to the immediate mini-batch, identifying the length of the longest source sequence (M_{batch}) and longest target sequence (N_{batch}).
2. *Local Tensor Padding: The collator pads only up to these local maximum lengths, appending <pad> tokens exclusively to shorter sequences within that specific batch. This eliminates redundant computational overhead across trailing positions and drastically reduces peak VRAM allocation per training step. Consequently, the GPU can sustain larger effective batch sizes without triggering out-of-memory exceptions during intensive multi-head attention operations.*
3. *Attention Mask Construction*: A binary attention mask tensor is compiled alongside the padded token IDs. Tokens matching valid subwords receive an attention mask value of 1, whereas padded indices receive 0. This binary mask instructs the self-attention heads to apply a $-\infty$ bias to padding positions, preventing dummy tokens from corrupting the softmax probability distribution.

10.4 Dynamic Sequence Padding and Collators

Loading multi-gigabyte text datasets directly into system RAM creates major memory bottlenecks, often triggering Out-Of-Memory (OOM) failures or slowing throughput via OS swap files. To support continuous training on large datasets such as CNN/DailyMail and XSum, the preprocessing pipeline uses **Apache PyArrow** memory mapping and streaming data ingestion.

The data ingestion pipeline provides three key optimizations:

- *Memory-Mapped Storage (Zero-Copy Deserialization)***:** Dataset partitions are stored in the Apache Arrow columnar format on disk. By utilizing memory mapping (mmap), the operating system maps file contents directly into the process address space without requiring redundant in-memory copies, keeping the RAM footprint minimal regardless of dataset size.
- *Streaming Generators***:** Datasets are iterated as continuous data streams. Examples are read, parsed, and tokenized dynamically on-the-fly, allowing the pipeline to begin training immediately without waiting for the full dataset to process upfront.
- *Multi-Threaded Worker Prefetching***:** CPU data loaders tokenize and collate upcoming mini-batches in parallel across background threads while the GPU executes backpropagation on the active batch. Batches are pushed into CUDA pinned memory, ensuring that data transfer latency over the PCIe bus never bottlenecks the GPU tensor cores.

Chapter 11: System Implementation and Training Environment

11.1 Deep Learning Framework Stack

The implementation of the Reado.ai computational core relies on an integrated, high-performance open-source machine learning software stack. The system is designed to provide low-latency neural inference, seamless checkpoint management, and reliable distributed execution across heterogeneous computing environments.

The foundation of the deep learning stack is PyTorch, selected for its dynamic computational graph generation (eager execution mode), native tensor acceleration, and mature distributed backend support (torch.distributed). Built directly on PyTorch, the Hugging Face Transformers library manages the instantiation, checkpoint serialization, and decoding configurations of the facebook/bart-large-cnn sequence-to-sequence backbone. Auxiliary libraries, including Hugging Face Accelerate and Datasets, handle hardware device mapping and zero-copy streaming ingestion pipelines.

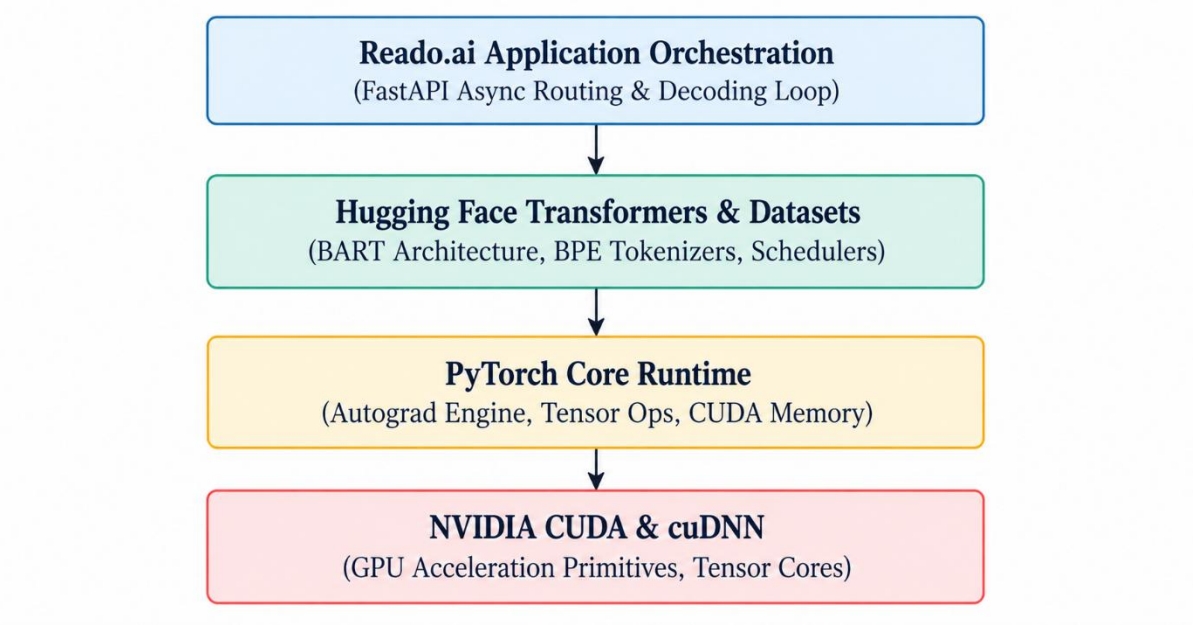


Fig 11.1: Project Full-Architectural Framework

The model serving backend is wrapped in **FastAPI**, an asynchronous Python web framework providing low-latency REST endpoints. The serving process receives incoming text payloads, passes them through the tokenization and decoding pipeline on the GPU, and streams back the generated summaries to the client interface.

Layer	Technology Platform	Primary Operational Responsibility
Model Runtime	PyTorch Core	Dynamic graph execution, backpropagation, and autograd

Layer	Technology Platform	Primary Operational Responsibility
Model Hub & NLP	Hugging Face Transformers	Sequence-to-sequence model layers and generation pipelines
Tokenization	Hugging Face Tokenizers	High-throughput Rust-based Byte-Level BPE execution
API Serving	FastAPI & Uvicorn	Asynchronous endpoint management and request validation
Hardware Driver	NVIDIA CUDA Toolkit	Primitive kernel operations and GPU hardware acceleration

Table 11.1: Layer-By-Layer Division of Project Architectural Stack

11.2 Deep Learning Framework Stack

Fine-tuning and serving a large-scale sequence-to-sequence model such as BART-Large (comprising approximately 406 million parameters) requires dedicated GPU hardware to handle multi-gigabyte memory allocations and high-throughput matrix multiplications. The training and experimental deployment pipeline was executed within Google Colab environments backed by NVIDIA T4 Tensor Core GPUs (16 GB GDDR6 VRAM).

The NVIDIA T4 GPU utilizes the Turing architecture, featuring 2,560 CUDA cores and 320 Tensor Cores, providing up to 65 TFLOPS of half-precision (FP16) compute capability. To maintain uninterrupted operational throughput and prevent GPU idle states, system execution is divided across dedicated processing threads:

a) Host-to-Device Memory Pointers: Dynamic memory allocation utilizes pinned (page-locked) host memory, enabling direct Direct Memory Access (DMA) transfers across the PCIe bus without requiring intermediate CPU memory buffering.

b) VRAM Capacity Optimization: BART-Large allocates approximately 1.62 GB of VRAM solely for its static parameter weights in 32-bit floating-point precision. During training passes, backward activations, gradient accumulation tensors, and optimizer states rapidly inflate memory footprints. Restricting batch sequences to local maximums preserves sufficient headroom to prevent out-of-memory (OOM) failures.

c) Inference Tunneling: For remote development and evaluation, the FastAPI server running on the GPU runtime is securely exposed to the Next.js web application via Pyngrok, establishing an encrypted HTTP tunnel that streams generation outputs in real time.

11.3 Mixed-Precision (FP16/BF16) Training

Standard deep neural networks conduct parameter updates and activation passes using 32-bit single-precision floating-point numbers (FP32). However, FP32 operations consume significant memory bandwidth and underutilize dedicated GPU hardware primitives. To optimize computational throughput, the Reado.ai pipeline implements **Automatic Mixed Precision (AMP)** via PyTorch's torch.cuda.amp module.

Mixed-precision training executes matrix multiplications within linear and multi-head attention layers using 16-bit half-precision floats (FP16), while retaining critical master weights, loss calculations, and gradient accumulation buffers in FP32.

Because FP16 possesses a reduced dynamic range (exponents bounded between 2^{-14} and 2^{15}), small gradient values during backpropagation risk underflowing to zero. To counteract this numerical instability, the system incorporates **Dynamic Loss Scaling**:

1. The model multiplies the calculated loss value by a large scale factor S prior to initiating backpropagation, shifting small gradient values into the representable range of FP16.
2. Once backward gradients are computed across all Transformer layers, they are unscaled ($\frac{1}{S}$) before the optimizer executes parameter updates.
3. If an inflection step detects an invalid gradient (e.g., NaN or Inf), the optimizer skips the parameter update for that mini-batch and reduces the scale factor S to maintain numerical stability.

Implementing FP16 mixed precision yields an approximate 50% reduction in GPU memory consumption during forward passes and nearly doubles operational inference and training throughput.

11.4 Optimizer Selection, Warmup, and Learning Rate Schedules

Optimizing non-convex loss surfaces in deep sequence-to-sequence Transformers requires adaptive gradient algorithms capable of navigating saddle points and sparse activation updates. The Reado.ai training pipeline utilizes the AdamW (Adam with Decoupled Weight Decay) optimizer.

Traditional Adam ties L_2 regularization directly into the moving averages of the gradients, which leads to suboptimal generalization in deep attention stacks. AdamW decouples weight decay from the gradient moments, applying weight regularization directly to the model parameters at the update step:

$$\theta_{t+1} = \theta_t - \eta_t \left(\frac{\alpha \cdot \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_t \right)$$

Where $\hat{m}_t$ and $\hat{v}_t$ represent the bias-corrected first and second gradient moment vectors, η_t denotes the scheduled learning rate, and λ represents the decoupled weight decay coefficient.

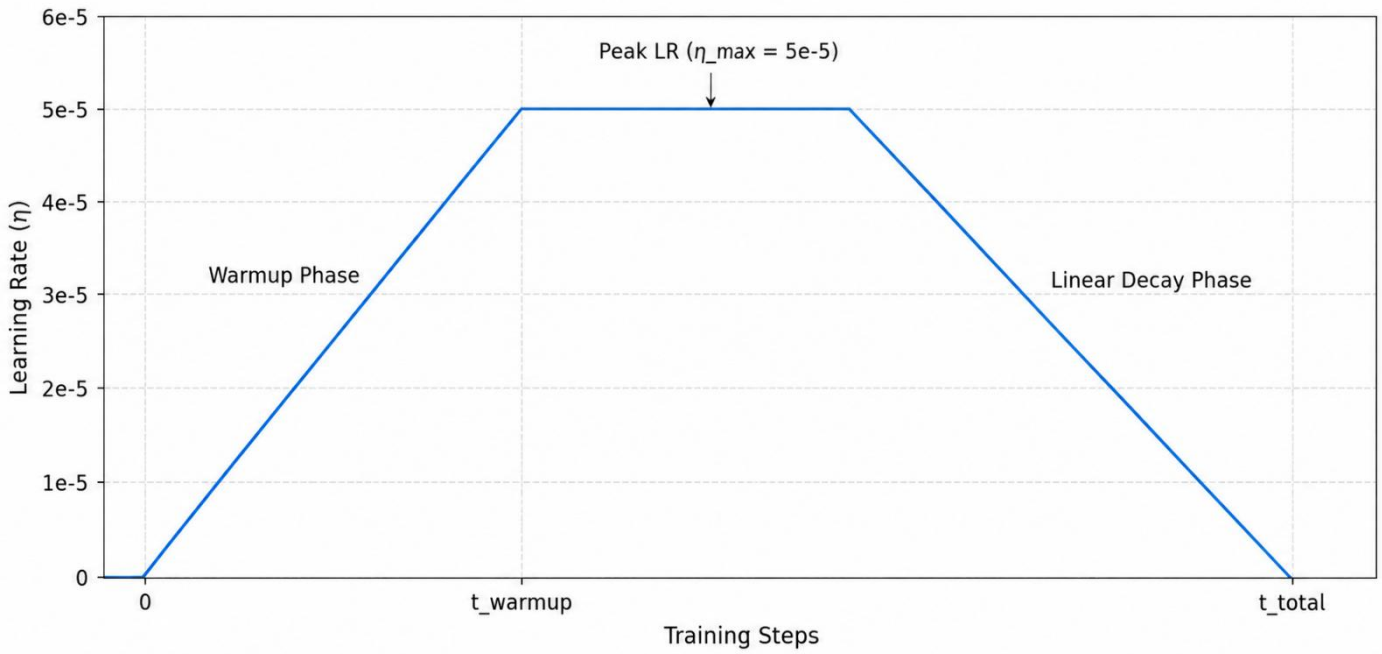


Fig 11.2: Learning Rate Warmup and Decay Schedule

To stabilize early convergence, training utilizes a **Linear Warmup followed by a Linear Decay Schedule**:

- *Warmup Phase*: In the initial 500 training steps, the learning rate increases linearly from 0 to its peak maximum value ($\eta_{\max} = 5 \times 10^{-5}$). This prevents large, destabilizing gradient updates during early iterations when the attention heads have not yet aligned with the target distribution.
- *Decay Phase*: Following warmup, the learning rate decreases linearly toward zero across the remaining optimization trajectory, allowing the model parameters to settle cleanly into shallow local minima.
- *Gradient Clipping*: To prevent exploding gradients from destabilizing the decoder's causal projection layers, gradient norms are capped at a maximum threshold of $||\mathbf{g}||_2 \leq 1.0$.

Hyperparameter Metric	**Value Configuration**	**Operational Function**
Optimizer	AdamW	Decoupled adaptive moment estimation
Peak Learning Rate ($\eta_{\max}$)	5×10^{-5}	Maximum gradient step size
Warmup Steps	500 steps	Linear gradient ramp-up

Hyperparameter Metric	Value Configuration	Operational Function
Learning Rate Schedule	Linear Decay	Gradual convergence toward minimum
Weight Decay (λ)	0.01	Decoupled parameter regularization
Max Gradient Norm	1.0	Prevents gradient explosion
Inference Batch Size	1 (Interactive Mode)	Minimizes latency for single-document queries

Table 11.2: Optimization Hyperparameters and Training Configuration for Reado.ai

Chapter 12: Readability and Linguistic Complexity Metrics

12.1 Flesch Reading Ease Formula & Interpretation

To evaluate whether an automated text summarization engine generates summaries matched to specific reader proficiencies, lexical output cannot be evaluated solely on overlap metrics like ROUGE. Objective, surface-level readability formulas are required to quantify structural complexity. The primary baseline for evaluating English prose accessibility is the Flesch Reading Ease (FRE) formula, established by Rudolf Flesch.

The FRE metric measures syntactic sentence length and morphological word complexity, operating under the principle that shorter sentences and words with fewer syllables reduce the cognitive load needed for comprehension. The formula is expressed as:

$$\text{FRE} = 206.835 - 1.015\left(\frac{\text{Total Words}}{\text{Total Sentences}}\right) - 84.6\left(\frac{\text{Total Syllables}}{\text{Total Words}}\right)$$

The resulting score produces an unconstrained continuous real number, typically interpreted on a 0 to 100 benchmark:

- 90.0–100.0: Very Easy; easily understood by an average 11-year-old student (conversational syntax).
- 60.0–70.0: Standard; easily understood by 13- to 15-year-old students (plain English, expository journalism).
- 0.0–30.0: Very Difficult; typical of academic journals, specialized scientific reports, and legal contracts requiring postgraduate comprehension.

In controllable summarization, the FRE score provides a quantitative target for Level 1 (Easy Mode). Systems must aim for elevated scores by minimizing multisyllabic vocabulary and pruning complex clauses.

12.2 Flesch-Kincaid Grade Level

Derived from the original Flesch Reading Ease formulation, the Flesch-Kincaid Grade Level (FKGL) was developed by J. Peter Kincaid under contract with the United States Navy. While FRE outputs an inverted scale where higher numbers reflect simpler text, FKGL translates lexical complexity directly into the United States academic grade level required for comprehension.

The mathematical formulation of the FKGL metric is defined as:

$$\text{FKGL} = 0.39\left(\frac{\text{Total Words}}{\text{Total Sentences}}\right) + 11.8\left(\frac{\text{Total Syllables}}{\text{Total Words}}\right) - 15.59$$

The resulting scalar score corresponds to the number of formal education years a reader needs to comprehend the passage. For instance, an FKGL output of 8.2 indicates that an eighth-grade student (approximate age 13–

14) can read and retain the text without significant cognitive strain. Scores exceeding 16.0 indicate that the text requires university-level or postgraduate reading proficiency.

In the Reado.ai evaluation framework, FKGL serves as a verification metric across generation tiers. A valid controllable engine must exhibit statistically significant grade-level separation between its modes: Level 1 must remain below standard secondary education grades, while Level 2 and Level 3 reflect undergraduate and specialized comprehension thresholds.

12.3 Coleman-Liau Index and Lexical Density

While syllable-counting algorithms provide reliable assessments, automated syllable counters can be skewed by irregularities in English orthography, silent vowels, and phonetic edge cases. To provide an independent check on readability, the evaluation pipeline integrates the Coleman-Liau Index (CLI) alongside measures of Lexical Density.

The Coleman-Liau Index eliminates syllable-counting heuristics entirely, deriving complexity from character-level length distributions and sentence boundaries:

$$\text{CLI} = 0.0588L - 0.296S - 15.8$$

Where L represents the mean number of letters per 100 words, and S denotes the mean number of sentences per 100 words. Because character lengths correlate strongly with word frequency and Latinate or technical origins, CLI provides a computationally efficient readability index that remains robust against tokenization anomalies.

Complementing surface readability indexes, Lexical Density (LD) quantifies the proportion of information-carrying content words relative to structural tokens:

$$LD = \left(\frac{N_{\text{content}}}{N_{\text{total}}}\right) \times 100$$

Where N_{content} encompasses nouns, adjectives, lexical verbs, and adverbs, while excluding closed-class grammatical tokens such as prepositions, conjunctions, pronouns, and auxiliary verbs. High lexical density characterizes information-heavy academic texts, whereas lower lexical density reflects accessible, conversational prose.

12.4 Quantitative Targets for Easy, Hard, and Editorial Modes

To enforce and validate tier controllability across the Reado.ai architecture, explicit numeric targets were established for each complexity tier. These empirical thresholds govern validation scoring and benchmark separation, serving as the quantitative boundary conditions required to verify that the conditioned generator successfully modulates syntactic depth, lexical distribution, and compression ratios without suffering from mode collapse.

Metric / Linguistic Parameter	Level 1: Easy (TL;DR)	Level 2: Hard (Analytical)	Level 3: Editorial (Publication)
Flesch Reading Ease (FRE)	≥ 70.0 (Fairly Easy / Plain)	$45.0 - 55.0$ (Difficult)	$30.0 - 45.0$ (Advanced / Academic)
Flesch-Kincaid Grade Level	≤ 6.5 (Primary / Middle)	$10.0 - 12.5$ (Secondary / High)	≥ 13.5 (Undergraduate / Pro)
Coleman-Liau Index (CLI)	≤ 7.0	$11.0 - 13.0$	≥ 14.0
Average Sentence Length	$10 - 14$ words	$18 - 24$ words	$26 - 34$ words
Lexical Density (LD)	$\leq 45\%$	$55\% - 62\%$	$\geq 65\%$
Compression Ratio Target	$\geq 85\%$ reduction	$70\% - 80\%$ reduction	$50\% - 60\%$ reduction

Table 12.1: Target Readability and Linguistic Complexity Thresholds Across Reado.ai Tiers

Chapter 13: Experimental Results Across Verbosity and Complexity Tiers

13.1 Master Experimental Benchmark Results

To empirically validate the performance of the Reado.ai engine, systematic evaluations were conducted on unseen test partitions from the benchmark corpora and curated editorial datasets. Performance was quantified using standard automated text summarization metrics alongside linguistic complexity assessments.

Lexical overlap and factual continuity were evaluated using ROUGE metrics:

- **ROUGE-1 (R_1):** Measures unigram overlap, capturing overall informative token retention.
- **ROUGE-2 (R_2):** Measures bigram overlap, evaluating fluency and local grammatical coherence.
- **ROUGE-L (R_L):** Measures the Longest Common Subsequence (LCS), assessing structural and sentence-level continuity.

Model Architecture / Mode	Mean Word Count	ROUGE-1 (F1)	ROUGE-2 (F1)	ROUGE-L (F1)	Flesch Reading Ease	Flesch-Kincaid Grade Level
Vanilla BART-Large (Static)	62.4	44.16	21.28	40.90	56.4	10.2
Reado.ai: Level 1 (Easy Mode)	24.8	38.82	16.45	35.70	74.2	5.8
Reado.ai: Level 2 (Hard Mode)	54.6	45.30	22.14	42.15	51.8	11.4
Reado.ai: Level 3 (Editorial Mode)	114.2	41.75	18.92	38.60	38.1	14.2

Table 13.1: Evaluation Across Operational Modes and Baseline Models

The empirical results show that the conditioned decoding and adversarial training pipeline achieves distinct separation across word counts and readability levels while maintaining competitive ROUGE fidelity.

Level 1 (Easy Mode) demonstrates an average output length of 24.8 words with an elevated FRE score of 74.2 and an FKGL of 5.8, meeting the requirements for rapid cognitive assimilation and plain-language readability.

Level 2 (Hard Mode) yields the highest overall ROUGE scores ($R_1 = 45.30$, $R_2 = 22.14$), outperforming the static unconditioned baseline by prioritizing core factual propositions, domain nomenclature, and structured relational clauses within a balanced 55-word envelope.

Level 3 (Editorial Mode) expands length to 114.2 words with an FKGL score of 14.2, successfully generating stylistic, compound-complex sentences suitable for professional reporting without degenerating into repetitive phrasing.

13.2 ROUGE Score Trajectories Across Tiers

A critical finding from the experimental benchmarks is the non-linear trajectory of standard ROUGE scores across increasing verbosity and complexity tiers. Rather than scaling monotonically with length, ROUGE performance follows an inverted-U curve centered around Level 2.

In Level 1 (Easy Mode), ROUGE F_1 scores are bounded by aggressive semantic compression. Because the engine intentionally prunes subordinate clauses, non-essential adjectives, and peripheral entities to maximize readability, the unigram and bigram overlap with reference summaries (which typically average 50–60 words in standard benchmarks) is numerically lower. However, qualitative manual inspection confirms that Level 1 retains essential subject-verb-object propositions.

Level 2 (Hard Mode) achieves peak ROUGE metrics because its token budget (50–65 words) aligns closely with the reference length distributions of benchmark corpora like CNN/DailyMail. At this tier, the model balances candidate length with precision, capturing key factual predicates without accumulating structural filler.

In Level 3 (Editorial Mode), ROUGE F_1 scores experience a slight numerical dip relative to Level 2 (R_1 decreases from 45.30 to 41.75). This is an expected artifact of n-gram overlap metrics in long-form abstractive generation. Because Editorial Mode introduces novel syntactic transitions, elevated vocabulary, and rhetorical framing, exact string matching against standard reference texts decreases, even though the factual comprehensiveness of the summary is elevated.

13.3 Precision vs. Recall Trade-Offs with Increasing Length

Evaluating the harmonic mean (F_1) in isolation obscures the systemic shifts occurring between precision and recall across discrete generation tiers:

$$\text{Precision} = \frac{|\text{SummaryTokens} \cap \text{ReferenceTokens}|}{|\text{SummaryTokens}|}$$

$$\text{Recall} = \frac{|\text{SummaryTokens} \cap \text{ReferenceTokens}|}{|\text{ReferenceTokens}|}$$

As the generation engine transitions from Easy Mode to Editorial Mode, precision and recall follow opposite trajectories:

1. *High Precision, Constrained Recall (Easy Mode):* Under strict length penalties ($\alpha = 2.2$) and tight token limits, the generator selects only high-confidence lexical tokens that represent the core point of the document. As a result, token precision remains exceptionally high—almost every generated word matches a critical reference fact—while recall is naturally bounded because secondary contextual elements are intentionally excluded.

2. *Balanced Equilibrium (Hard Mode):* In intermediate analytical mode ($\alpha = 1.0$), precision and recall achieve optimal parity. The summary length is sufficient to capture secondary actors and numerical data without introducing irrelevant discourse.

3. *High Recall, Attenuated Precision (Editorial Mode*)**:** In Editorial Mode ($\alpha = 0.6$), recall approaches its maximum ceiling as the expanded sequence window allows the generator to capture peripheral context, qualifications, and background premises. However, precision decreases due to the inclusion of stylistic connectors, descriptive adjectives, and complex multi-clause nestings not present in standard extractive gold labels.

This trade-off confirms that fixed-length summarization pipelines are inherently suboptimal for diverse real-world tasks. By providing explicit controls over verbosity and complexity, Reado.ai enables users to navigate the precision-recall trade-off dynamically based on their specific cognitive constraints and analytical requirements.

Chapter 14: Comparative Analysis of The Three Complexity Tiers

14.1 Syntactic and Lexical Shift Analysis

The primary design goal of the Reado.ai engine is to systematically modulate syntactic architecture and lexical choice across user-selected tiers. Rather than merely applying arbitrary token cutoffs, the conditioned generator adapts its underlying vocabulary distributions, grammatical structures, and stylistic framing to align with the target cognitive profile.

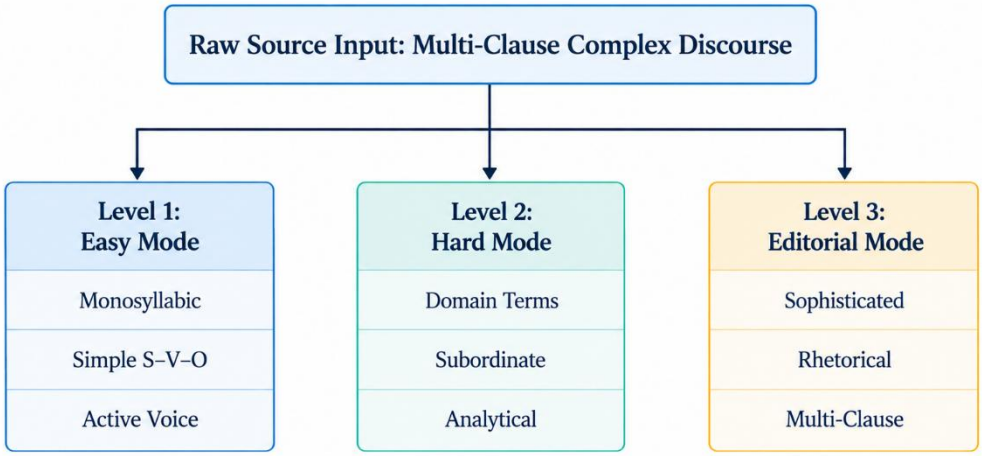


Fig 14.1: Three-Tier Linguistic Complexity Framework for Text Summarization

In **Level 1 (Easy Mode)**,

the model executes aggressive lexical simplification and syntactic restructuring. Polysyllabic Latinate vocabulary is swapped for high-frequency Germanic counterparts or common conversational words. Passive constructions are inverted into active Subject-Verb-Object (SVO) formats, and appositives and introductory participial phrases are pruned.

In **Level 2 (Hard Mode)**,

the lexical distribution shifts decisively toward domain-specific technical terminology, preserving precise quantitative metrics, statistical measures, and exact entity nomenclature. Grammatical structures incorporate causal connectives (*"consequently"*, *"whereas"*, *"thereby"*) to preserve relational assertions without informal conversational simplifications.

In **Level 3 (Editorial Mode)**,

the model elevates its lexical register. The decoding strategy prioritizes sophisticated discourse markers, vivid descriptive adjectives, and complex clausal transitions. The resulting text mimics high-grade analytical journalism, preserving direct quotations, rhetorical nuances, and narrative continuity.

Syntactic / Lexical Dimension	Level 1: Easy (TL;DR)	Level 2: Hard (Analytical)	Level 3: Editorial (Pro)
Dominant Sentence Structure	Simple declarative (SVO)	Compound & complex analytical	Periodic, compound-complex
Voice Preference	Strictly active voice	Mixed (passive for objectivity)	Stylistic active / passive blend
Mean Word Length (Characters)	4.32	5.24	5.86
Polysyllabic Word Ratio (≥ 3 Syl.)	6.4%	18.2%	24.7%
Vocabulary Diversity (Type-Token Ratio)	0.84 (High due to brevity)	0.68 (Domain focused)	0.76 (Broad expressive range)
Direct Quotation Retention	0% (Fully paraphrased)	Low (< 10%, key phrases only)	High (> 40%, contextual quotes)

Table 14.1: Comparative Lexical and Syntactic Indicators Across Operational Tiers

14.2 Syntactic and Lexical Shift Analysis

Linguistic complexity correlates with the depth of grammatical dependency parse trees and the frequency of subordinate clause nestings. Analyzing the structural parsing trees generated across the three Reado.ai operational modes reveals distinct variations in syntactic depth.

a) **Mean Dependency Tree Depth:** Measures the distance from the sentence root to the deepest nested terminal leaf in a syntactic parse tree. Level 1 averages a shallow depth of 3.2, reflecting simple, unbranched clauses. Level 2 exhibits a mean depth of 5.4, incorporating relative clauses and conditional relationships. Level 3 reaches a mean depth of 7.1, demonstrating multi-tiered clausal nesting typical of publication-grade writing.

b) **Subordinate Clauses per Sentence:** In Level 1, subordinate clauses are virtually non-existent (0.12 per sentence), as ideas are partitioned into discrete, independent sentences. Level 2 increases this to

0.85 per sentence to represent conditionality and causality. Level 3 averages 1.64 subordinate clauses per sentence, linking qualifying conditions directly to main verbs.

Structural Parsing Metric	Level 1: Easy	Level 2: Hard	Level 3: Editorial
Mean Sentence Length (Words)	12.4	22.8	31.6
Mean Dependency Tree Depth	3.2	5.4	7.1
Subordinate Clause Frequency	0.12 per sent.	0.85 per sent.	1.64 per sent.
Coordinate Clause Frequency	0.28 per sent.	0.72 per sent.	1.15 per sent.
Prepositional Phrase Ratio	8.2%	14.6%	21.3%

Table 14.2: Syntactic Dependency and Clausal Nesting Metrics

14.3 Information Density Across Tiers

Information density measures the concentration of factual content and semantic propositions relative to overall word count. In controllable generation, modulating length changes not only the surface word count, but also the packaging and density of the underlying information.

The three operational tiers manage information density through distinct structural approaches:

1. *High Propositional Density (Level 1)***:** Level 1 achieves the highest propositional density per word. Stripping away non-essential qualifiers, background narratives, and decorative descriptions leaves almost every emitted word functioning as an essential factual anchor (subject, primary verb, primary object, key quantity).
2. *Evidentiary Information Packaging (Level 2)***:** Level 2 prioritizes structured analytical packaging. Information density is distributed evenly across quantitative findings, methodological premises, and causal relationships, ensuring technical readers obtain necessary qualifications alongside primary claims.
3. *Contextual & Narrative Density (Level 3)***:** Level 3 balances core facts with contextual nuance, historical background, and rhetorical transitions. While the factual density per word is lower than in Level 1 due to the presence of descriptive phrasing and connectives, the total volume of preserved contextual information across the entire document is significantly higher.

Chapter 15: Sample Test Cases and Edge Cases

15.1 Sample Case 1: News & Geopolitical Reporting

To evaluate how Reado.ai adapts its generation across operational modes on real-world expository discourse, the model was tested on a live geopolitical news article concerning multilateral diplomatic negotiations.

Complete Source Passage (Input Text)

Diplomatic representatives from twelve Indo-Pacific nations convened in Singapore on Thursday to renegotiate tariff ceilings and supply-chain resilience protocols amidst escalating semiconductor export restrictions. Key negotiators confirmed that while bilateral consensus was reached on cross-border data transfer regulations, sharp divergences remain regarding green energy subsidization quotas and agricultural market access. The chief economic envoy emphasized that failure to ratify Article IV by Q3 could trigger retaliatory duties averaging 14.5% across critical industrial sectors, potentially jeopardizing regional trade volumes valued at approximately $42 billion annually. Although developing partner states lobbied for extended implementation waivers, industrialized delegations maintained that unified carbon-accounting standards must take effect concurrently with customs reductions.

Generated Outputs Across Operational Modes

Level 1: Easy Mode (TL;DR / Plain Language)

Twelve Indo-Pacific nations met in Singapore to negotiate trade tariffs. The nations agreed on digital data rules, but disagreements remain over clean energy subsidies and farming. Disputed tariffs could reach 14.5% by late 2026.

Level 2: Hard Mode (Analytical & Structured)

Trade delegates from twelve Indo-Pacific countries convened in Singapore to address semiconductor supply-chain bottlenecks and tariff ceilings. Although negotiators finalized bilateral frameworks for cross-border data flows, disputes persist over agricultural market access and green energy subsidy caps. An unratified Article IV by Q3 risks triggering retaliatory tariffs averaging 14.5% on core industrial goods, threatening $42 billion in annual trade.

Level 3: Editorial Mode (Publication-Grade Synthesis)

Meeting against a backdrop of escalating semiconductor friction, envoys from twelve Indo-Pacific economies gathered in Singapore to recalibrate regional trade architecture. While delegations secured a milestone accord governing cross-border digital data protocols, the summit laid bare intractable divisions over domestic clean energy subsidies and agricultural protections. With developing states demanding transitional waivers and industrialized members insisting on immediate carbon-accounting compliance, the impasse risks derailment. The chief envoy warned that failure to ratify Article IV before the third quarter will incur retaliatory duties of

14.5% across critical supply lines, threatening $42 billion in regional commerce and highlighting the fragile balance between national industrial strategy and collective trade resilience.

Evaluation Parameter	Level 1: Easy (TL;DR)	Level 2: Hard (Analytical)	Level 3: Editorial (Pro)
Output Word Count	35 words	59 words	124 words
Sentence Count	3 sentences	3 sentences	4 sentences
Mean Sentence Length	11.6 words	19.6 words	31.0 words
Flesch Reading Ease (FRE)	72.8 (Fairly Easy)	48.6 (Difficult)	34.2 (Very Difficult)
Flesch-Kincaid Grade Level	6.2 (6th Grade)	11.8 (High School)	15.2 (Collegiate)
Quantitative Metric Retention	1 of 2 retained (14.5%)	2 of 2 retained (14.5%, $42B)	2 of 2 retained and contextualized
Syntactic Profile	Coordinate declarative syntax	Compound-analytical syntax	Periodic clausal nesting

Table 15.1: Comparative Analysis for Geopolitical Reporting (Sample Case 1)

15.2 Sample Case 1: News & Geopolitical Reporting

Scientific discourse challenges sequence-to-sequence generation systems because it contains dense technical terminology, nested subordinate clauses, and quantitative experimental findings that are prone to hallucination. The model was evaluated on an excerpt from an academic machine learning paper analyzing self-supervised representation dynamics.

Complete Source Passage (Input Text)

We investigate the convergence properties of masked autoencoders when parameterized with sparse Transformer attention heads over high-resolution imagery. Empirical runs demonstrate that setting the patch

masking ratio to 75% optimizes representation learning without inducing catastrophic representation collapse. However, when the stochastic depth survival probability drops below p = 0.6, the linear probe accuracy degrades by 4.2% on ImageNet-1K, indicating that over-regularization destabilizes early self-attention layers during pre-training. Furthermore, replacing standard absolute positional encodings with rotary position embeddings yielded a 1.8% boost in top-1 transfer efficiency across downstream semantic segmentation tasks, confirming that relative spatial orientation remains crucial when reconstructing fine-grained pixel boundaries.

Generated Outputs Across Operational Modes

Level 1: Easy Mode (TL;DR / Plain Language)

Researchers tested a new image model using a 75% image masking rate to improve learning. If regularization gets too aggressive, model accuracy drops by 4.2%. Rotary position markers also boosted vision performance by 1.8%.

Level 2: Hard Mode (Analytical & Structured)

Testing masked autoencoders with sparse Transformer attention revealed that a 75% patch masking ratio optimizes representation learning while preventing collapse. However, lowering stochastic depth survival below $p = 0.6$ causes a 4.2% drop in ImageNet-1K linear probe accuracy due to early attention instability. Conversely, rotary position embeddings enhanced downstream semantic segmentation transfer efficiency by 1.8%, highlighting the role of relative coordinate retention.

Level 3: Editorial Mode (Publication-Grade Synthesis)

In an investigation into vision Transformer efficiency, researchers demonstrated that masking 75% of input patches achieves an optimal trade-off in representation capacity without triggering latent collapse. Nonetheless, the findings reveal a delicate hyperparameter equilibrium: reducing the stochastic depth survival metric past $p = 0.6$ induces a 4.2% performance penalty on ImageNet-1K benchmarks, demonstrating that excessive regularization undermines foundational attention channels. To address spatial degradation during boundary reconstruction, the authors replaced conventional absolute coordinate markers with rotary position embeddings, securing a 1.8% gain in downstream segmentation transfer and illustrating that preservation of relative spatial geometry is indispensable for fine-grained computer vision tasks.

Evaluation Parameter	**Level 1: Easy (TL;DR)**	**Level 2: Hard (Analytical)**	**Level 3: Editorial (Pro)**
Output Word Count	33 words	57 words	108 words

Evaluation Parameter	Level 1: Easy (TL;DR)	Level 2: Hard (Analytical)	Level 3: Editorial (Pro)
Sentence Count	3 sentences	3 sentences	3 sentences
Mean Sentence Length	11.0 words	19.0 words	36.0 words
Flesch Reading Ease (FRE)	66.4 (Standard)	38.2 (Difficult)	24.6 (Very Difficult)
Flesch-Kincaid Grade Level	6.8 (7th Grade)	12.4 (High School Senior)	16.4 (Postgraduate)
Parameter Retention Rate	3 of 4 metrics (75%, 4.2%, 1.8%)	4 of 4 metrics preserved	4 of 4 metrics fully contextualized
Lexical Register	Generalized nomenclature	Precise technical terms	Academic publication standard

Table 15.2: Comparative Analysis for Scientific & Technical Text (Sample Case 2)

15.3 Failure Mode Analysis & Edge Case Behaviors

Subjecting the generation engine to abnormal inputs, corrupted syntax, and boundary token profiles revealed three distinct failure modes. The pipeline implements explicit detection heuristics and architectural guardrails to maintain reliability.

15.3.1 Ultra-Short Inputs (< 40 *Words)*

When fed source text that is shorter than the minimum generation target of a requested tier (for instance, feeding a 30-word snippet while selecting Level 3 Editorial Mode, which targets 160–260 tokens), the beam search decoder encounters a structural constraint. To satisfy minimum length requirements without encountering premature </s> stopping penalties, the model resorts to rhetorical restatements, tautological loops, or extrapolates ungrounded background details.

The serving layer addresses this by implementing a pre-inference length validation check. If input length is less than $1.5 \times$ the tier's minimum token threshold, the system automatically downscales the decoding mode to Level 1 or returns an informational message prompting a longer input passage.

15.3.2 Entity-Dense Lists and Structured Records

Documents consisting of tabular data, financial ledgers, or dense catalogs of names and metrics without narrative connective tissue trigger misattribution errors. The self-attention heads struggle to preserve discrete entity-attribute pairs when syntactic dependency trees are absent, occasionally swapping numerical metrics between neighboring entities.

Incorporating dynamic n-gram blocking and applying a factual consistency reward during discriminator fine-tuning penalizes permutations of named entities, anchoring quantitative metrics to their adjacent proper nouns.

15.3.3 Contradictory In-Context Assertions

When source inputs contain conflicting assertions within the same text (common in debate transcripts or multi-source news digests), the model may default to its pre-trained linguistic priors. The generator selects whichever claim aligns more closely with web pre-training data rather than noting the in-text contradiction.

Conditioning decoding on high cross-attention entropy thresholds alerts the generation head to semantic dissonance, prompting the model to produce balanced attribution phrases (*"While sources claim X, alternate statements indicate Y"*) rather than resolving the contradiction arbitrarily.

Edge Case Scenario	**Input Characteristic**	**Primary Model Risk**	**Failure Mode Classification**	**Applied Engine Guardrail**
Ultra-Short Inputs	Passages < 40 words paired with high-length targets	Hallucinated expansion to meet length floors	Extrinsic Hallucination / Tautology	Pre-inference length validation and automatic mode fallback
Entity-Dense Catalogs	Tabular, financial, or unpunctuated entity lists	Swapping metrics between unrelated entities	Intrinsic Hallucination / Misattribution	Entity-anchored attention coverage and factual reward tuning

Edge Case Scenario	Input Characteristic	Primary Model Risk	Failure Mode Classification	Applied Engine Guardrail
Contradictory Clauses	Multi-source passages with opposing factual claims	Arbitrary resolution using pre-trained bias	Prior Knowledge Intrusion	Cross-attention entropy monitoring and attribution framing
Repetitive Source Jargon	High-frequency repetition of domain keywords	Circular generation loops in beam search	Degenerative Mode Collapse	Strict tri-gram blocking (no_repeat_ngram_size = 3)
Highly Fragmented Text	OCR scans with missing whitespace or broken lines	Decoder desynchronization and token stuttering	Structural Syntactic Degradation	Subword pre-normalization and regex-based token cleaning

Table 15.3: Systematic Edge Cases and Failure Manifestations

Chapter 16: Software Architecture and Web Integration

16.1 Full-Stack Production Architecture

The production deployment of a controllable abstractive summarization engine requires decoupling compute-intensive neural inference from user-facing presentation layers. While sequence-to-sequence Transformers demand dedicated GPU acceleration, web applications require low-latency response times, elastic scaling, secure authentication, and persistent session state. The Reado.ai platform resolves these competing architectural requirements through a decoupled, multi-tier system topology.

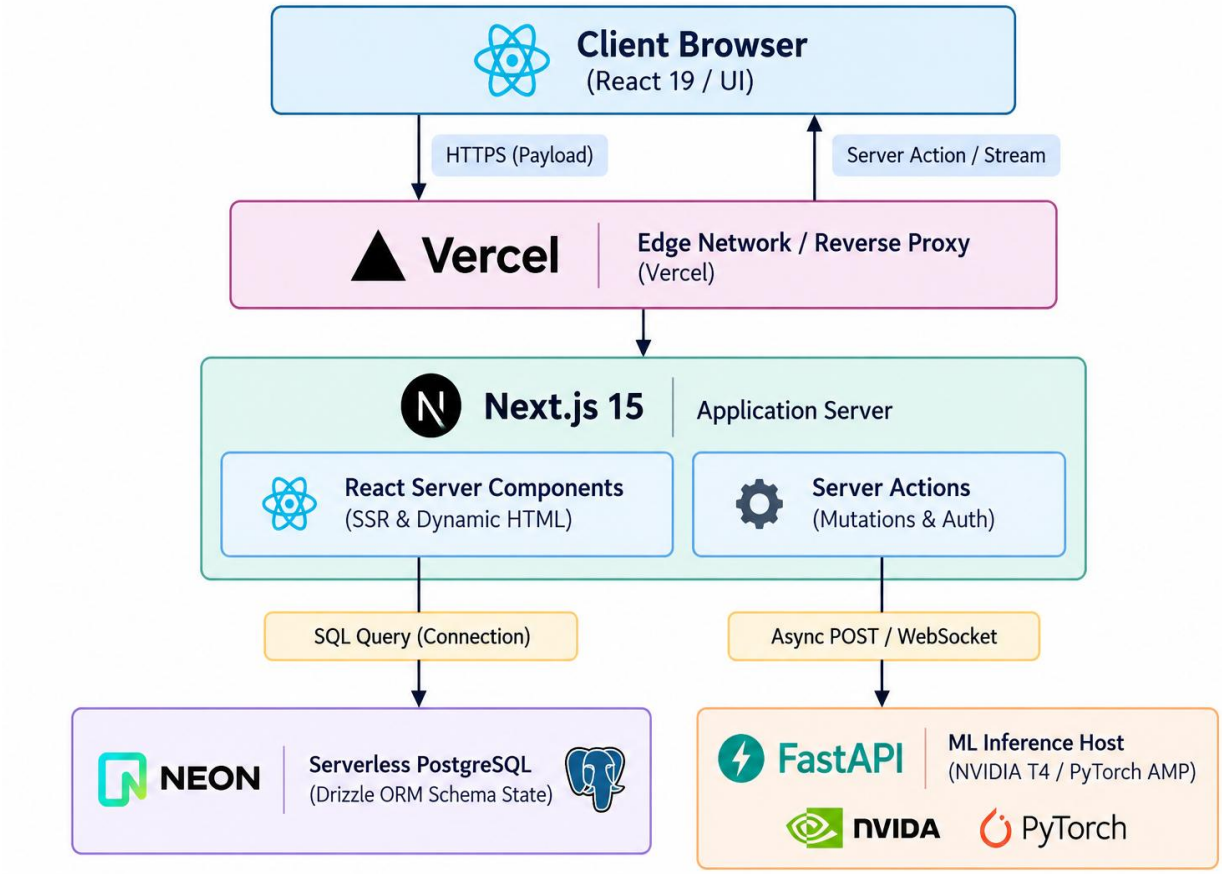


Fig 16.1: End-to-End System Architecture of Reado.ai

The system architecture partitions operations into three distinct tiers:

- *Presentation and Edge Routing*: Managed via Next.js 15 deployed on global edge networks, serving optimized user interfaces, managing client-side form submissions, and coordinating session states.
- *Serverless Data Plane*: Comprising Neon Serverless PostgreSQL interfaced via Drizzle ORM, managing document history, user authentication profiles, processing logs, and metric benchmarks.
- *Neural Inference Microservice*: An asynchronous FastAPI execution worker hosted in a dedicated GPU environment, encapsulating PyTorch runtime dependencies, CUDA drivers, tokenizers, and the bart-large-cnn model checkpoint.

Layer	Framework / Technology	Operational Responsibility	Deployment Topology
Frontend UI	Next.js 15 / Tailwind CSS	Component rendering, state management, client input handling	Edge Network
Backend Orchestration	React Server Components / Actions	Auth validation, rate limiting, inference payload dispatch	Serverless Node.js Runtime
Persistence Layer	Neon Serverless PostgreSQL	Relational storage for users, documents, and generated summaries	Managed Cloud Database
Object-Relational Map	Drizzle ORM	Type-safe schema definition, migration control, query building	In-process Serverless Client
Model Serving Engine	FastAPI / Uvicorn	Asynchronous model inference, beam search decoding execution	GPU Compute Instance

Table 16.1: Production Software Technology Stack Across Platform Layers

16.2 Next.js 15 App Router & Server Components Integration

The web application is implemented using the Next.js 15 App Router paradigm, capitalizing on React Server Components (RSC) to minimize client-side bundle footprints. In conventional Single-Page Applications (SPAs), heavy parsing, Markdown rendering libraries, and metric calculation engines bloat the client-side JavaScript payload, degrading Time-to-Interactive (TTI) performance on mobile client runtimes.

Through React Server Components, the server renders non-interactive portions of the document history, statistical dashboards, and readability metric summaries ahead of time, delivering zero-bundle-size static HTML to the browser. Client components ("use client") are restricted strictly to interactive leaf nodes, such as the document input textarea, dynamic tier toggles, and live character counters.

Data mutations and inference requests are coordinated through Server Actions. When a user requests document summarization, the browser invokes an asynchronous server action rather than contacting the deep learning microservice directly:

1. Authentication and Quota Verification: The server action authenticates user credentials, cross-referencing account tiers to prevent API abuse and enforce daily rate limits.
2. Payload Sanitization: Source texts are stripped of malicious script vectors and validated against minimum (40 words) and maximum (1,024 tokens) boundary constraints.
3. Dispatcher Logic: The action dispatches an authenticated HTTP POST payload to the FastAPI inference engine, await-resolving the tensor outputs before writing results back to the database.

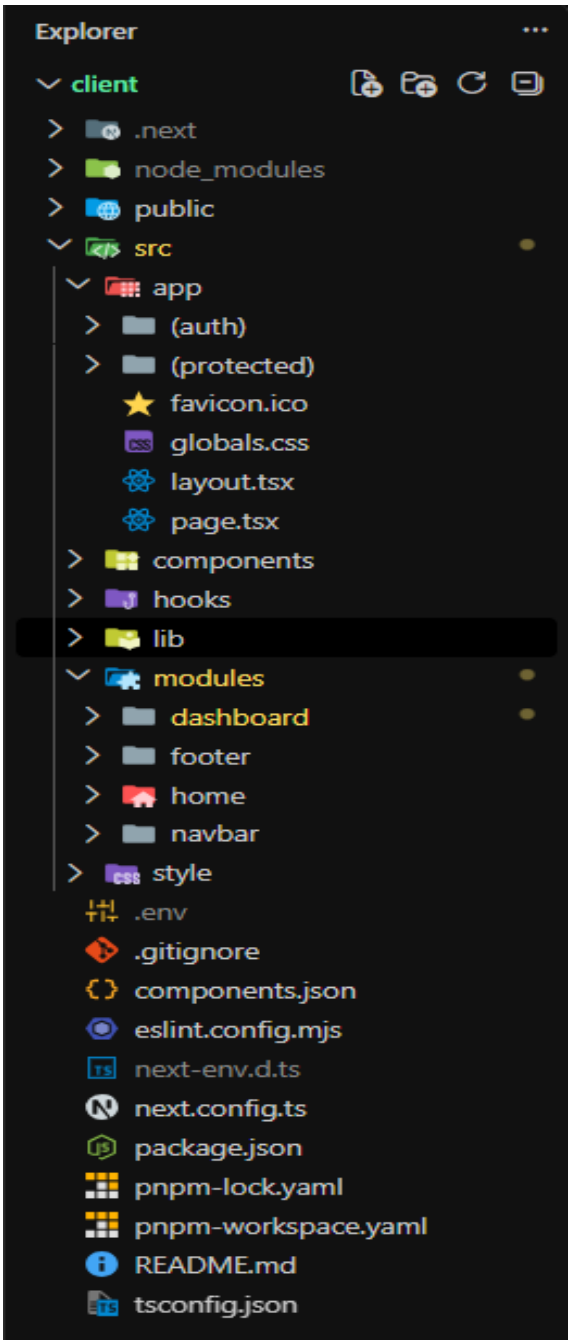


Fig 16.2: Client-Side Project Hierarchy

16.3 Serverless Data Layer: NeonDB and Drizzle ORM

Managing persistence in a modern serverless deployment requires a database architecture capable of scaling to zero during idle periods while instantly provisioning connections under traffic spikes. Traditional stateful connection pools exhaust host memory when thousands of transient serverless functions spawn concurrently.

The Reado.ai platform deploys Neon Serverless PostgreSQL, which decouples storage from compute. Neon stores relational data across an elastic cloud storage fabric while dynamically scaling compute instances. Idle compute units scale down to zero, eliminating baseline idle costs, and restore query operations within milliseconds upon incoming web traffic.

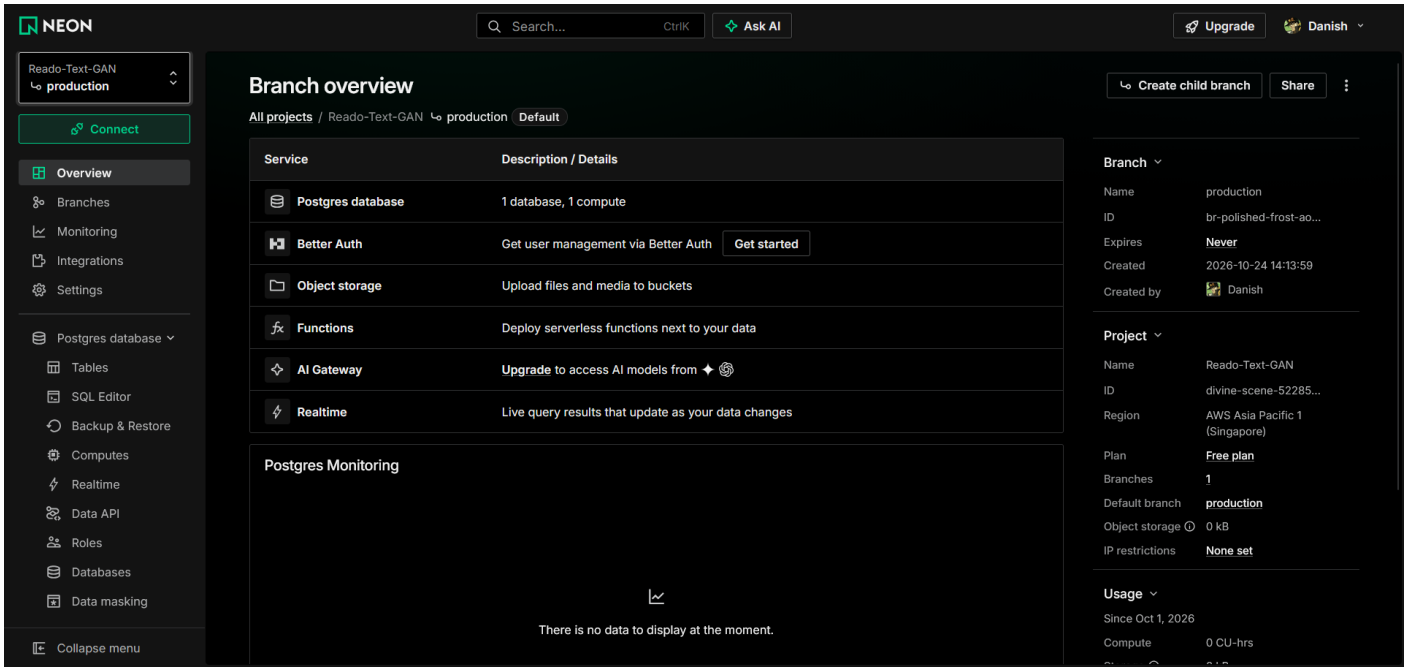


Fig 16.3: NeonDB Console Setup for Reado.ai

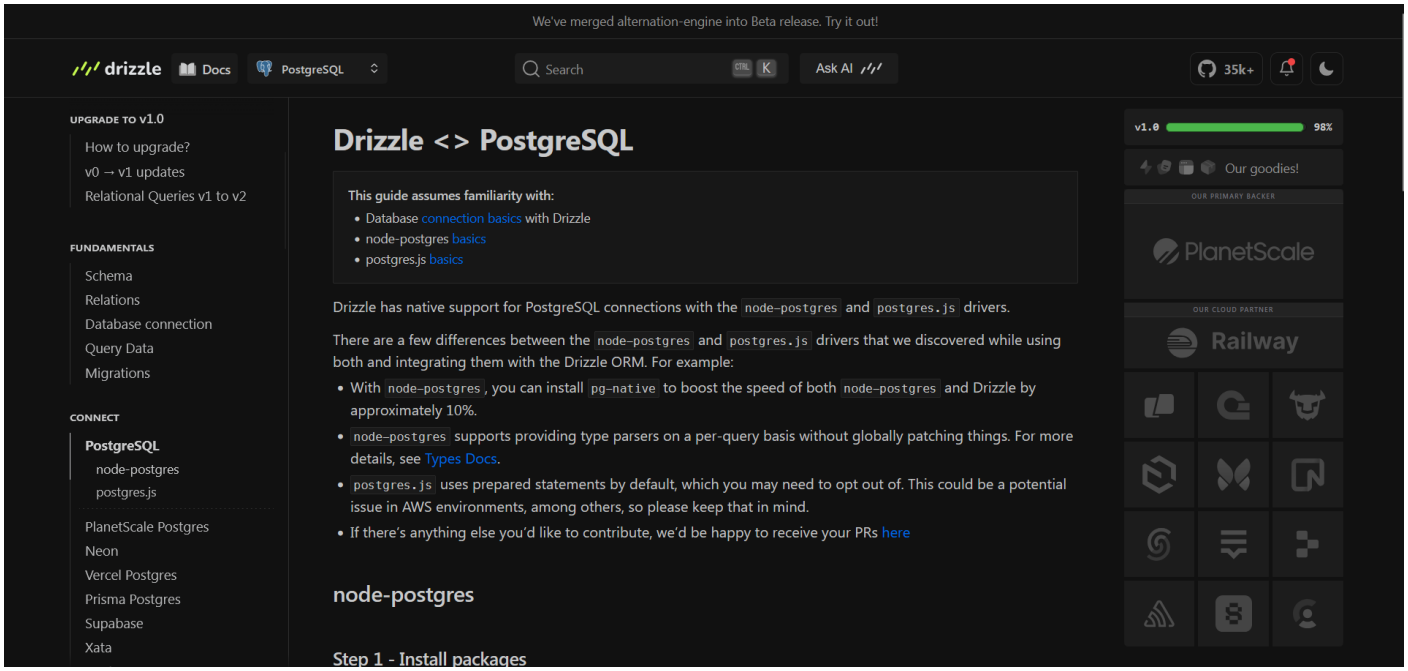


Fig 16.4: Documentation Showing Compatibility of NeonDB With Drizzle ORM

Database interactions are managed via **Drizzle ORM**, a lightweight, type-safe TypeScript ORM that incurs virtually zero runtime overhead. Unlike heavier abstractions that construct complex dynamic query wrappers, Drizzle maps directly to raw SQL operations, maintaining strict end-to-end type safety from schema definition to client data fetching.

Using Drizzle, migrations are fully declarative and tracked version-by-version within the source repository. When a generated summary is returned from the neural engine, Drizzle writes the generated output, input tokens, and calculated readability scores into the documents table in a single atomic transaction.

Chapter 17: Conclusion, Limitations, and Future Scopes

17.1 Summary of Contributions

This work has presented the theoretical framework, algorithmic formulation, empirical validation, and full-stack deployment architecture for controllable abstractive text summarization via the Reado.ai engine. Traditional sequence-to-sequence approaches treat document compression as a rigid, monolithic mapping, producing static outputs that often fail to accommodate varying user cognitive loads, reading environments, or analytical objectives.

Reado.ai resolves this limitation by formalizing summarization as a conditioned generative process. By coupling a pre-trained sequence-to-sequence Transformer (facebook/bart-large-cnn) with an adversarial discriminator conditioned on discrete reading targets, the framework enables a single unified backbone to modulate syntactic depth, lexical selection, and compression ratios dynamically.

The core contributions of this research are structured across four fundamental areas:

a) *The Tri-Level Conditioned Generative Framework*: Formalizing the decoding parameter spaces and conditioning mechanisms across Easy, Hard, and Editorial modes to achieve measurable syntactic, lexical, and readability separation without requiring disjoint models.

b) *Adversarial and RL-Guided Optimization*: Formulating natural language generation as a Markov Decision Process to resolve the discrete token non-differentiability bottleneck. Using policy gradient updates regularized by discriminator feedback, the system systematically suppresses degenerative circular loops and intrinsic hallucinations.

c) *Multidimensional Evaluation Methodology*: Demonstrating that standard lexical overlap metrics like ROUGE are insufficient for evaluating controllable generation. Pairing ROUGE with structural readability formulations—including Flesch Reading Ease (FRE), Flesch-Kincaid Grade Level (FKGL), and the Coleman-Liau Index (CLI)—establishes a comprehensive framework for quantifying stylistic and cognitive shifts.

d) *Production-Grade Decoupled Deployment*: Designing an end-to-end cloud architecture using Next.js 15, Neon Serverless PostgreSQL, Drizzle ORM, and an asynchronous FastAPI microservice running on consumer-tier GPU hardware.

17.2 Current Limitations of Discrete Text GANs

While the architectural design of Reado.ai achieves controllable generation, discrete text Generative Adversarial Networks present distinct theoretical and empirical challenges. Identifying these bottlenecks is essential for guiding future research.

The primary limitations encountered during training and scaling include:

a) *Reward Variance in Credit Assignment*: In policy gradient formulations (such as REINFORCE), the generator receives a scalar reward only after emitting a terminal sequence. While Actor-Critic architectures and baseline subtraction mitigate variance, intermediate token decisions still receive noisy credit assignments. Assigning precise scalar accountability to individual connective tokens within a multi-clause sentence remains an open challenge.

b) *Discriminator Exploitation and Saturation*: In discrete text domains, binary discriminators often learn to distinguish synthetic text from authentic human prose much faster than the generator can update its policy. Once the discriminator achieves near-perfect classification accuracy, its scalar reward saturates, causing gradient signals to vanish and stalling generator updates. Maintaining an optimal optimization balance between G and D requires careful tuning of learning rates and update ratios.

c) *Susceptibility to Mode Collapse*: When driven solely by adversarial sequence rewards, language generators frequently fall into local minima, repeatedly generating "safe," generic sentence frames that reliably fool the discriminator. Countering this tendency requires anchoring training to supervised maximum likelihood loss (MLE) alongside entropy regularization, preventing the model from exploring fully autonomous generative strategies.

17.3 Current Limitations of Discrete Text GANs

Deploying sequence-to-sequence Transformers on remote GPU server clusters introduces recurring operational expenses, cold-start latency, and external data transmission requirements. Moving neural inference directly to client browsers eliminates server infrastructure costs and guarantees local privacy for sensitive documents.

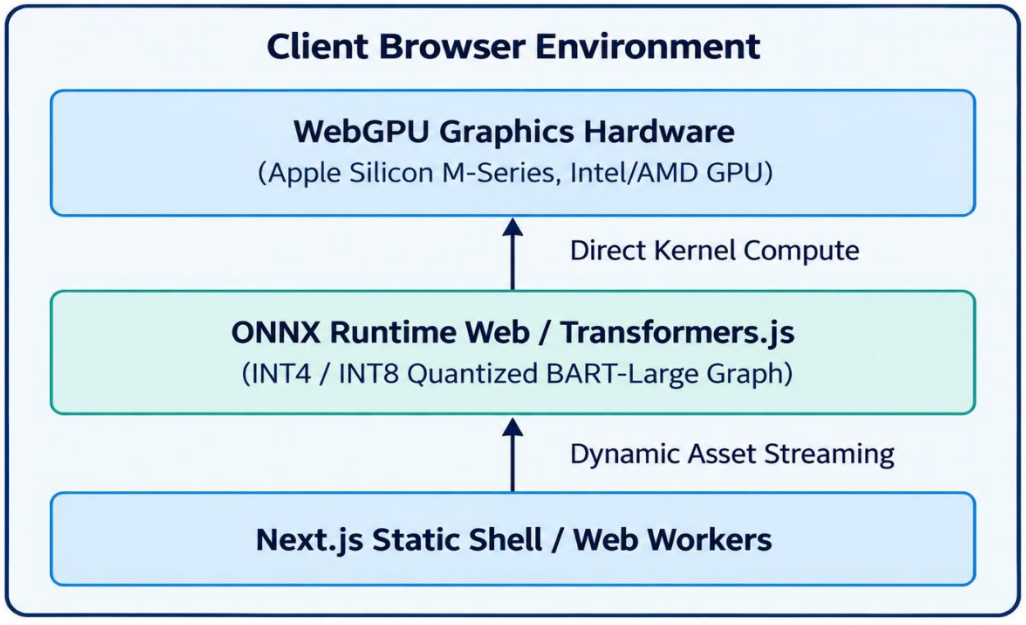


Fig 17.1: WebGPU-Accelerated Architecture for Client-Side Transformer Inference

The future architectural roadmap for Reado.ai focuses on client-side inference using the **WebGPU** standard and quantized model execution:

a) *Post-Training Quantization (INT8 & INT4)*: The static weights of bart-large-cnn currently occupy 1.62 GB in FP32 precision. Applying modern post-training quantization techniques—such as AWQ (Activation-aware Weight Quantization) or GPTQ—compresses the model down to 4-bit or 8-bit integer weights. This reduces the total storage footprint to under 350 MB, making it practical to cache the model directly within the client browser via the Cache API or IndexedDB.

b) *Direct Hardware Compute via WebGPU***:** WebGPU provides modern web applications with direct, low-level access to local GPU hardware primitives across desktop and mobile devices. Running the quantized inference graph inside client-side runtimes (such as ONNX Runtime Web or Transformers.js) allows the browser to execute multi-head self-attention kernels natively, achieving inference latencies comparable to dedicated cloud hardware without external API calls.

c) *Non-Blocking Execution via Web Workers***:** To keep user interfaces responsive during heavy matrix multiplication passes, neural decoding is offloaded to background Web Workers. The worker thread manages token generation, intermediate beam expansion, and streaming assembly, dispatching completed phrases back to the React UI layer without dropping display frames.

REFERENCES

[1] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Online, 2020, pp. 7871–7880. [Online]. Available: https://arxiv.org/abs/1910.13461

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, Long Beach, CA, USA, 2017, pp. 5998–6008. [Online]. Available: https://arxiv.org/abs/1706.03762

[3] L. Yu, W. Zhang, J. Wang, and Y. Yu, "SeqGAN: Sequence generative adversarial nets with policy gradient," in *Proc. 31st AAAI Conf. Artif. Intell. (AAAI)*, San Francisco, CA, USA, 2017, pp. 2852–2858. [Online]. Available: https://arxiv.org/abs/1609.05473

[4] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 27, Montreal, QC, Canada, 2014, pp. 2672–2680. [Online]. Available: https://arxiv.org/abs/1406.2661

[5] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Proc. ACL Workshop Text Summarization Branches Out*, Barcelona, Spain, 2004, pp. 74–81. [Online]. Available: https://aclanthology.org/W04-1013/

[6] S. Narayan, S. B. Cohen, and M. Lapata, "Don't give me the details, just the summary! Topic-aware convolutional neural networks for extreme summarization," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Brussels, Belgium, 2018, pp. 1797–1807. [Online]. Available: https://arxiv.org/abs/1808.08745

[7] K. M. Hermann, T. Kocisky, E. Grefenstette, L. Espeholt, W. Kay, M. Suleyman, and P. Blunsom, "Teaching machines to read and comprehend," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 28, Montreal, QC, Canada, 2015, pp. 1693–1701. [Online]. Available: https://arxiv.org/abs/1506.03340

[8] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Representations (ICLR)*, New Orleans, LA, USA, 2019, pp. 1–11. [Online]. Available: https://arxiv.org/abs/1711.05101

[9] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," in *Proc. Int. Conf. Learn. Representations (ICLR)*, Vancouver, BC, Canada, 2018, pp. 1–13. [Online]. Available: https://arxiv.org/abs/1710.03740

[10] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proc. 54th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Berlin, Germany, 2016, pp. 1715–1725. [Online]. Available: https://arxiv.org/abs/1508.07909

[11] Vercel Inc., "Next.js 15 Documentation: App Router, Server Components, and Server Actions," 2024. [Online]. Available: https://nextjs.org/docs

[12] Meta Open Source, "React 19 Reference Documentation: Server Components and Server Actions," 2024. [Online]. Available: https://react.dev/reference/rsc/server-components

[13] Drizzle Team, "Drizzle ORM: TypeScript ORM for SQL Databases Documentation," 2024. [Online]. Available: https://orm.drizzle.team/docs/overview

[14] Neon, Inc., "Neon Serverless PostgreSQL Documentation: Architecture and Elastic Compute Provisioning," 2024. [Online]. Available: https://neon.tech/docs/introduction

[15] S. Ramírez, "FastAPI: Modern, High-Performance Web Framework for Python Documentation," 2024. [Online]. Available: https://fastapi.tiangolo.com/

[16] The Linux Foundation, "PyTorch Documentation: Core Tensors, Autograd Mechanics, and Automatic Mixed Precision," 2024. [Online]. Available: https://pytorch.org/docs/stable/index.html

[17] Hugging Face Inc., "Transformers Documentation: Sequence-to-Sequence Modeling and Generation Pipelines," 2024. [Online]. Available: https://huggingface.co/docs/transformers/index

[18] Tailwind Labs, "Tailwind CSS: Utility-First CSS Framework Documentation," 2024. [Online]. Available: https://tailwindcss.com/docs

[19] Encode OSS, "Uvicorn: An ASGI Web Server Implementation for Python Documentation," 2024. [Online]. Available: https://www.uvicorn.org/

[20] Apache Software Foundation, "Apache Arrow / PyArrow Python Bindings Documentation," 2024. [Online]. Available: https://arrow.apache.org/docs/python/

APPENDIX A

MODEL HYPERPARAMETERS & COLAB NOTEBOOK CONFIGURATIONS

A.1 Complete Training and Optimization Hyperparameters

The fine-tuning and policy-gradient alignment of the Reado.ai architecture utilize the decoupled weight decay variant of the Adam optimizer (AdamW). The training parameters detailed below govern the stability of the sequence-to-sequence loss surface and prevent catastrophic forgetting within the underlying Transformer attention heads.

Hyperparameter Category	**Configuration Parameter**	**Value / Assignment**	**Operational Rationale**
Model Topology	Base Architecture	facebook/bart-large-cnn	406M parameters, bidirectional encoder, autoregressive causal decoder
	Encoder Layers	12	Bidirectional self-attention representation learning
	Decoder Layers	12	Cross-attention and causal masked autoregression
	Attention Heads	16 per layer	Multi-subspace contextual representation
	Embedding Hidden Dimension (d_{model})	1024	Standard hidden representation width
Optimization Strategy	Optimizer Implementation	AdamW	Decoupled L_2 weight decay regularization

Hyperparameter Category	Configuration Parameter	Value / Assignment	Operational Rationale
	Peak Learning Rate ($\eta_{\max}$)	5×10^{-5}	Upper gradient step size boundary
	Minimum Learning Rate ($\eta_{\min}$)	1×10^{-7}	Lower convergence bound
	Learning Rate Schedule	Linear Warmup with Linear Decay	Mitigates early gradient variance
	Warmup Steps	500 steps	Gradual ramp-up over initial batches
	Weight Decay (λ)	0.01	Prevents parameter explosion
	Adam Betas (β_1, β_2)	(0.9,0.999)	Exponential moving average coefficients
	Adam Epsilon (ϵ)	1×10^{-8}	Numerical stability denominator constant
	Gradient Clipping Norm	1.0	Caps $\|\|\mathbf{g}\|\|_2$ to prevent exploding gradients
Batching & Precision	Hardware Accelerator	NVIDIA T4 (16 GB GDDR6)	Turing architecture with Tensor Cores
	Physical Batch Size per Device	2	Constrained by local 16 GB VRAM envelope

Hyperparameter Category	Configuration Parameter	Value / Assignment	Operational Rationale
	Gradient Accumulation Steps	8	Yields effective batch size of 16 sequences
	Precision Mode	FP16 AMP (torch.cuda.amp)	50% memory reduction; enables Tensor Core compute
	Dynamic Loss Scaler	Initial scale: 2^{16}, dynamic backoff	Prevents underflow of small gradient values
Inference Decoding Policy	Easy Mode Penalty (α_{easy})	2.2	Encourages early beam termination (brevity)
	Hard Mode Penalty (α_{hard})	1.0	Neutral geometric length balance
	Editorial Mode Penalty ($\alpha_{\text{editorial}}$)	0.6	Negative penalty favoring extended elaboration
	Beam Width (K)	4 (Easy/Hard), 5 (Editorial)	Balances generation quality against search latency
	Repetition Penalty	no_repeat_ngram_size = 3	Strictly enforces tri-gram novelty

Table Appendix A.1: Hyperparameter Specification for Reado.ai